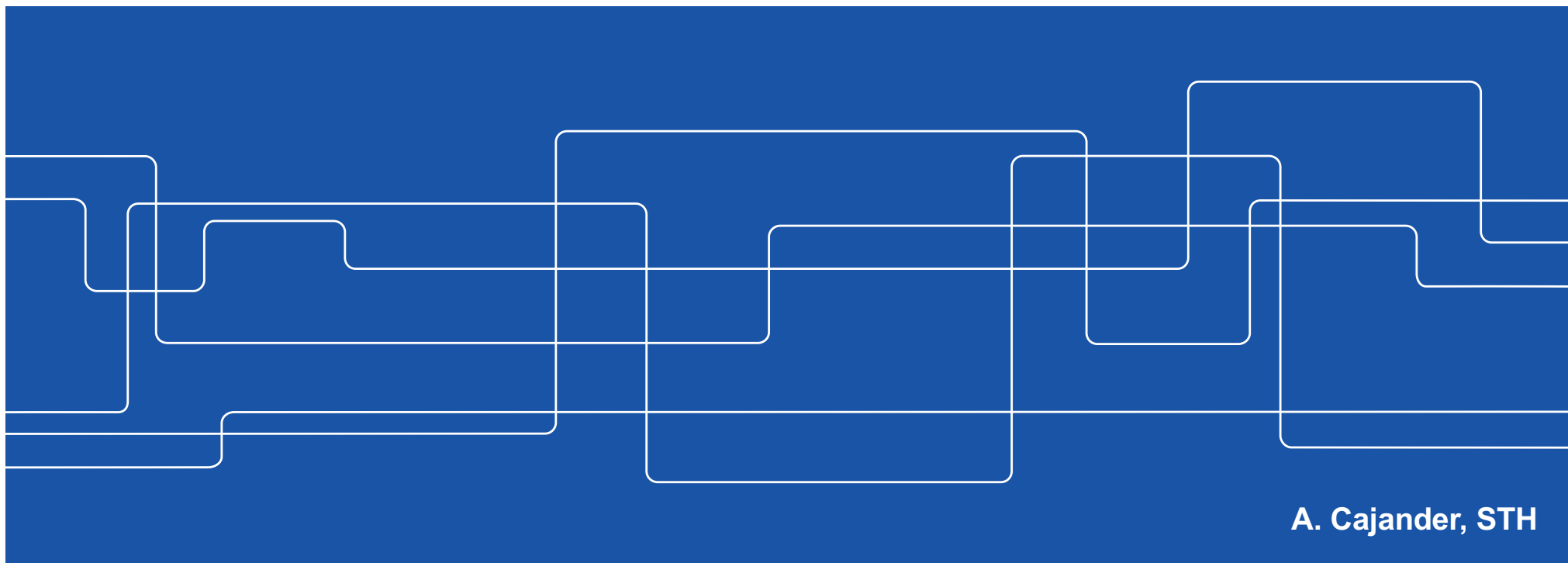




HE1025 Operativsystem

F7: Lagring – Storage



A. Cajander, STH



Kursen...

F1: V#1 Processorn.	Kap (1) 2..4 & 6
Ö1: V#2 c Pekare...	Kap 14 & c-boken
F2: V#3 Schemaläggning	Kap 7..9,(10-11), RTOS
L1: V#4 FreeRTOS	Lab #1 Handledning
F3: V#5 Segmentation	Kap 12..13, 15..17
Ö2: V#6 Buffer	t.b.d
F4: V#7 Paging	Kap 18-22, (23-24)
L2: V#8 Paging	Lab #2 Handledning
F5: C#1 Locks & CV	Kap (25) 26..27 & 28..29.1
Ö3: C#2 LINUX	Kap 5 (39)
F6: C#3 Semaphores	Kap 30..33, (34)
L3: C#4 Barberare	Lab #3 Handledning
F7: P#1 Storage	Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)
Ö4: P#2 DMA/SD Card	Exemplifiering av F7 tekniker
F8: P#3 Communication	Kap (47, 39) & 48, (49..51)
L4: P#4 Client/Server	Lab #4 Handledning

TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

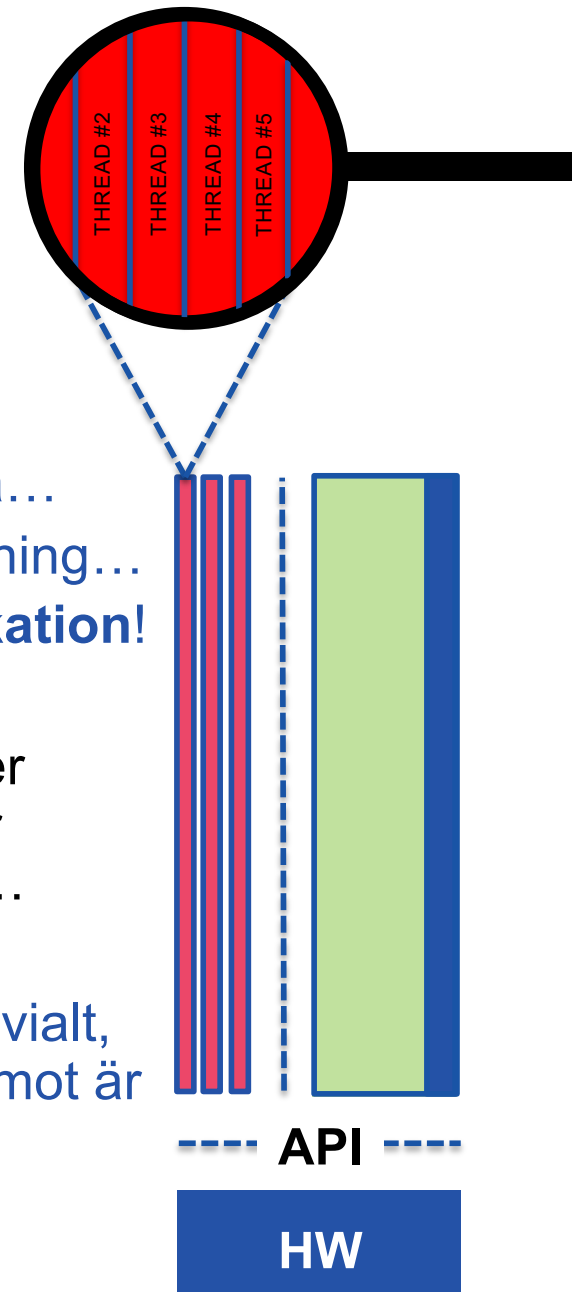
LAB1: Närvaro L1..4

Persistens & Storage

Bra, operativsystemets processer och trådar...
...har tillgång till minne och processor(er)...
...på ett effektivt sätt, men vi behöver också...
...erbjuda access till ansluten kringutrustning...
...för **icke flyktig lagring & kommunikation!**

Även om dylik utrustning blivit snabbare så sker datautbytet en faktor 100 till 1 000 000 000 ggr långsammare än datautbytet i en modern cpu...

...att managera ett sådant datautbyte är inte trivialt, utan att hela systemet blir långsammare. Däremot är själva gränssnittet rätt lika från enhet till enhet!





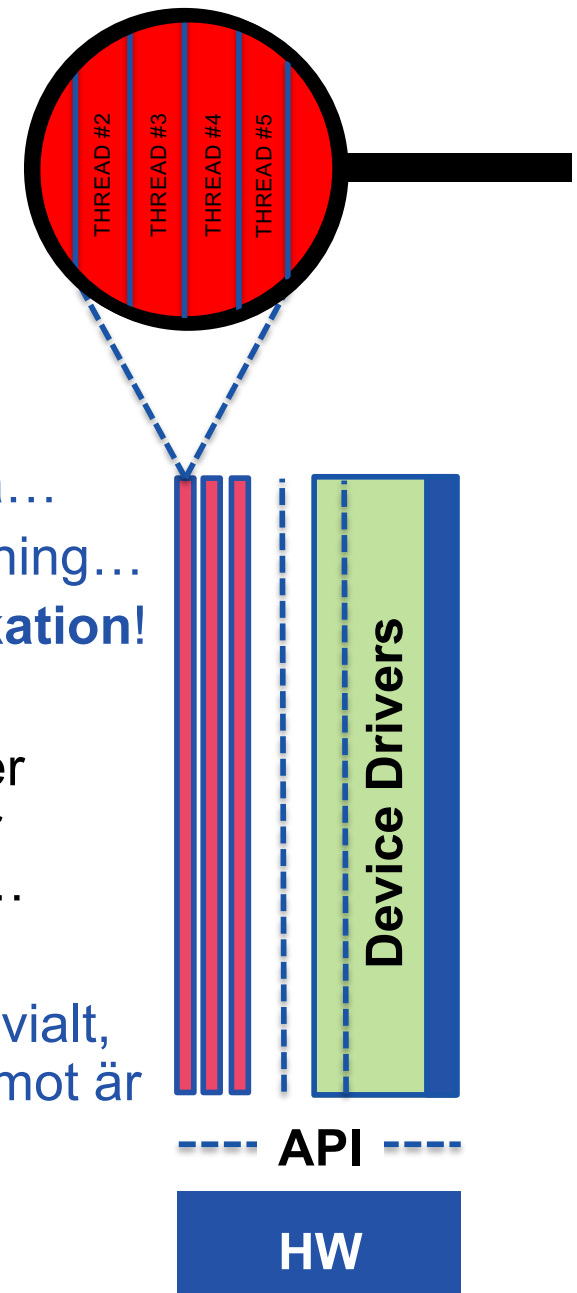
* Och skrivs oftast inte av samma leverantör som OS:et!

Persistens & Storage

Bra, operativsystemets processer och trådar...
...har tillgång till minne och processor(er)...
...på ett effektivt sätt, men vi behöver också...
...erbjuda access till ansluten kringutrustning...
...för **icke flyktig lagring & kommunikation!**

Även om dylik utrustning blivit snabbare så sker datautbytet en faktor 100 till 1 000 000 000 ggr långsammare än datautbytet i en modern cpu...

...att managera ett sådant datautbyte är inte trivialt, utan att hela systemet blir långsammare. Däremot är själva gränssnittet rätt lika från enhet till enhet!



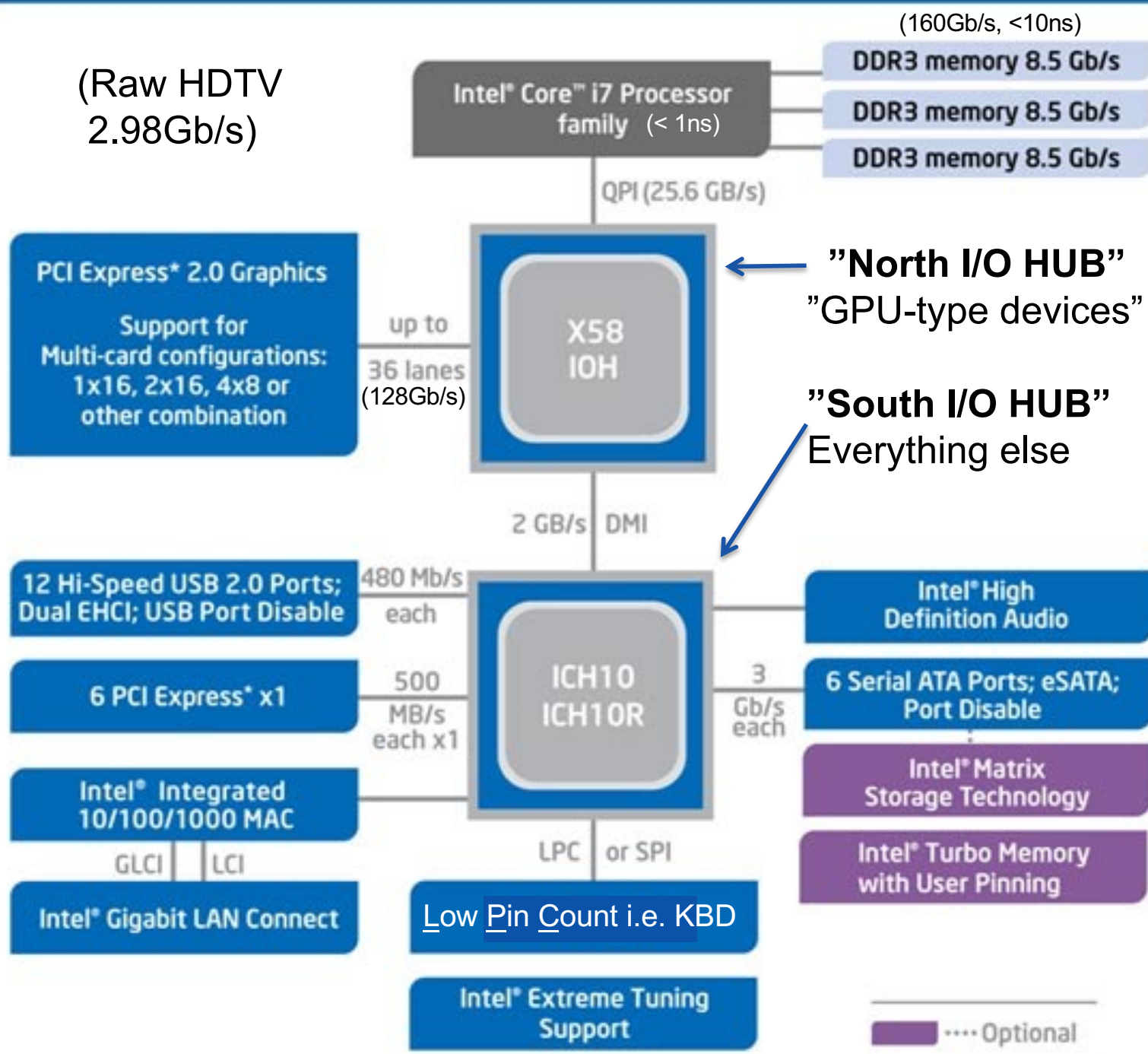
OS:et ger access till ansluten kringutrustning via så kallade Device Drivers, de utgör i dag ofta mer än 70% av koden* i ett os!



Systemarkitektur...

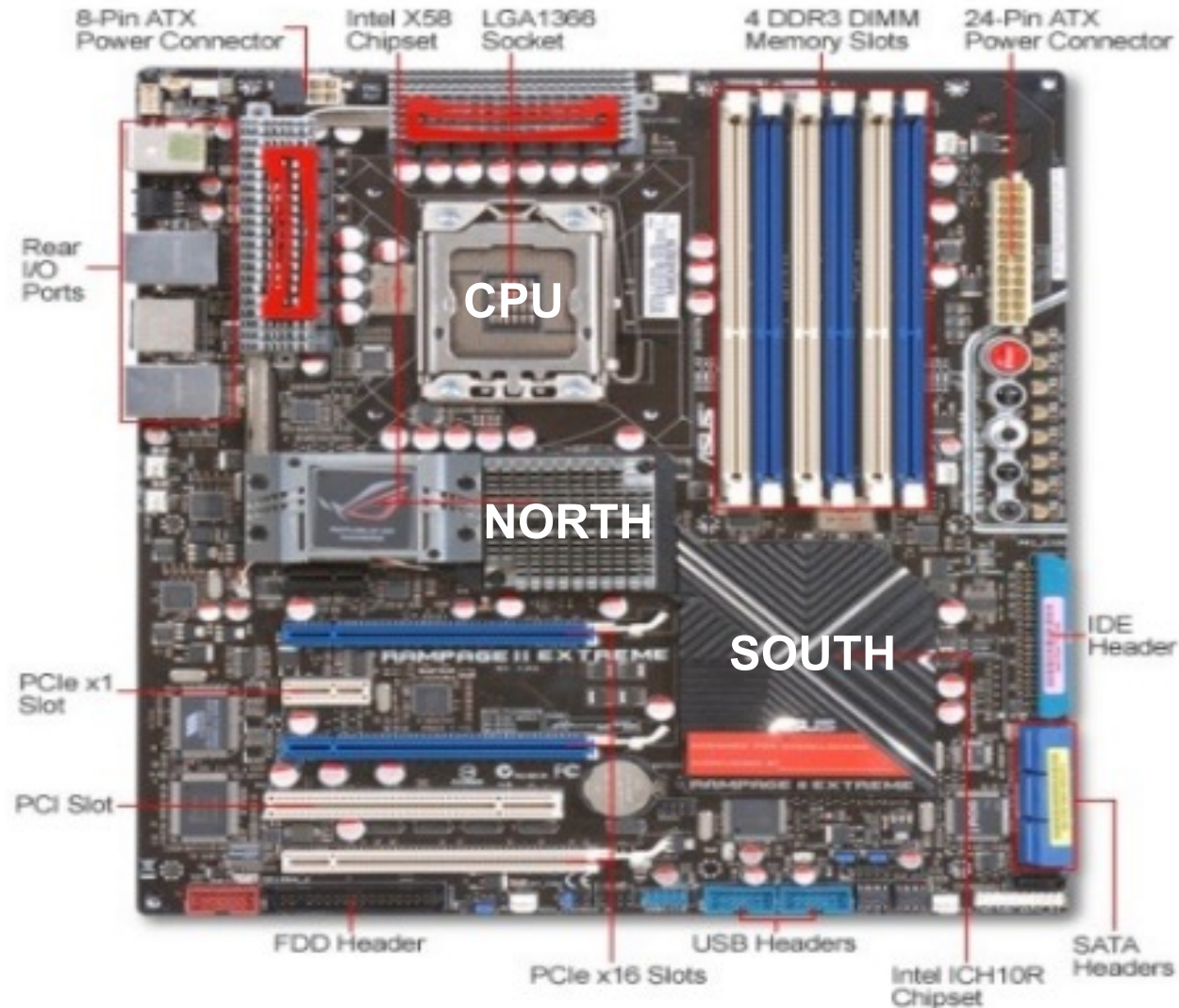
Ett antal device anslutna via väl spec. hårdvaru-gränssnitt!

(Raw HDTV
2.98Gb/s)



The Asus Rampage II Extreme

Top View





...via laddad
device driver!

4.1. Device Structs

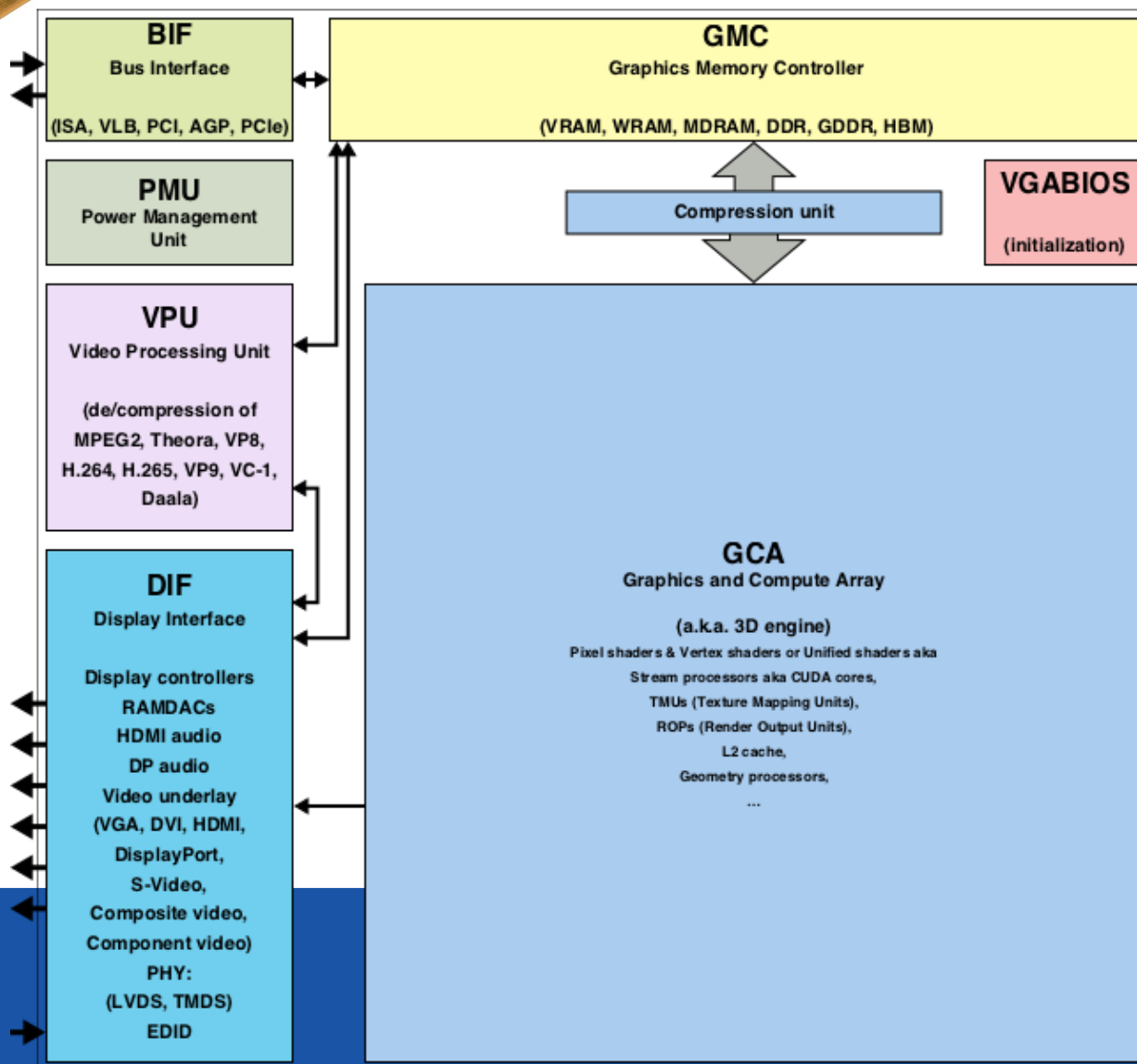
Classes

```
struct nvmlBAR1Memory_t
struct nvmlBridgeChipHierarchy_t
struct nvmlBridgeChipInfo_t
struct nvmlEccErrorCounts_t
struct nvmlMemory_t
struct nvmlNvLinkUtilizationControl_t
struct nvmlPciInfo_t
struct nvmlProcessInfo_t
struct nvmlSample_t
struct nvmlUtilization_t
union nvmlValue_t
struct nvmlViolationTime_t
```

(som alltså skapas
av kortleverantören.)

PCI-Express HW I/F...

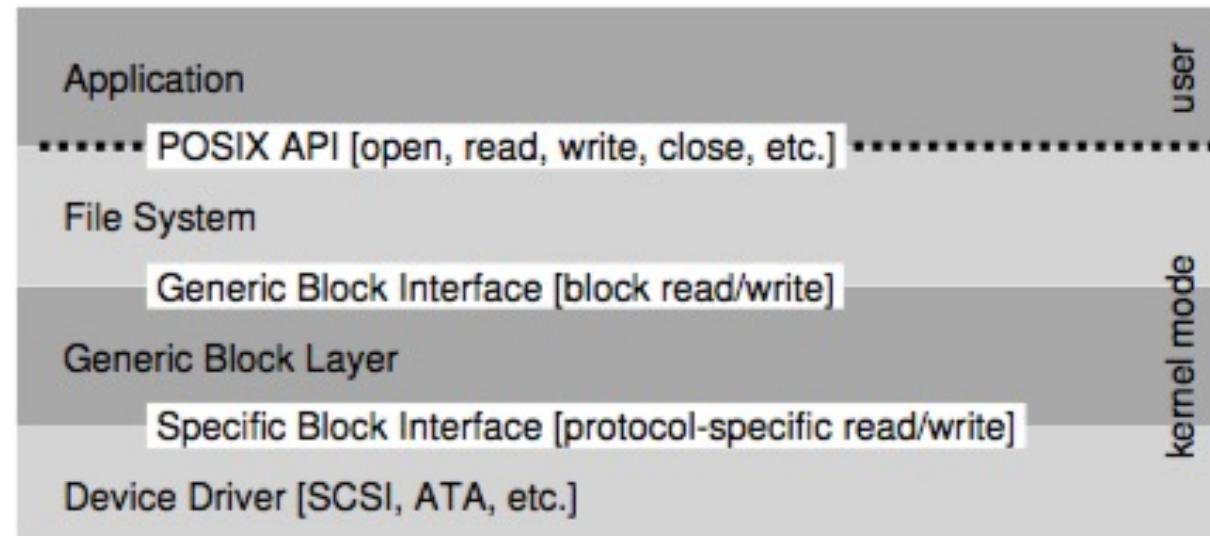
...ger access till enheten...





Systemarkitektur...

En process/tråd kan ofta kommunicera direkt med en specifik device driver men vanligtvis tillhör den anslutna hårdvaran med sin device driver någon typisk klass av enheter som vanligtvis återfinns i ett datorsystem...



...programmeraren når då hårdvaran via ett/flera lager av programvara som skapar ett generiskt gränssnitt till alla hårdvara en viss typ! (Vår vän abstraktion.)

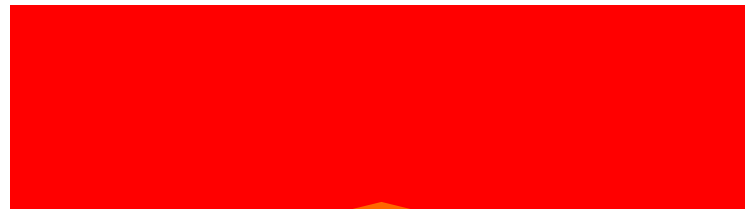
...det är inte trivialt att skriva en device driver, det är inte heller trivialt hur os:et ska kommunicera med ett device effektivt...



OS

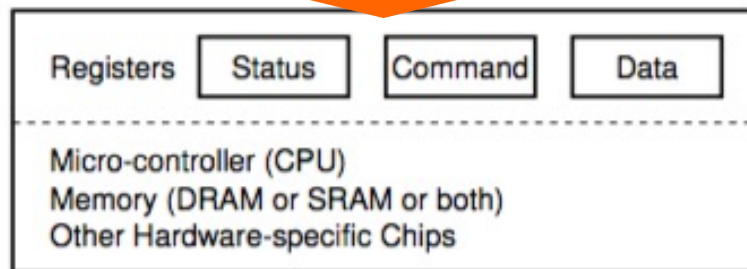
Vi tittar på ett generaliserat HD I/F...

File System/Generic Block I/F

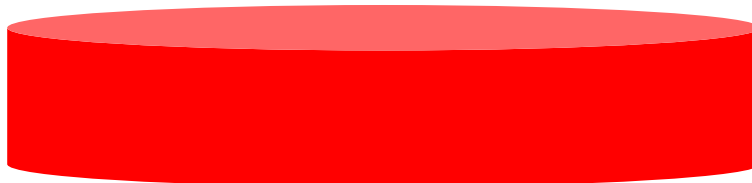


Laddad device driver

SATA



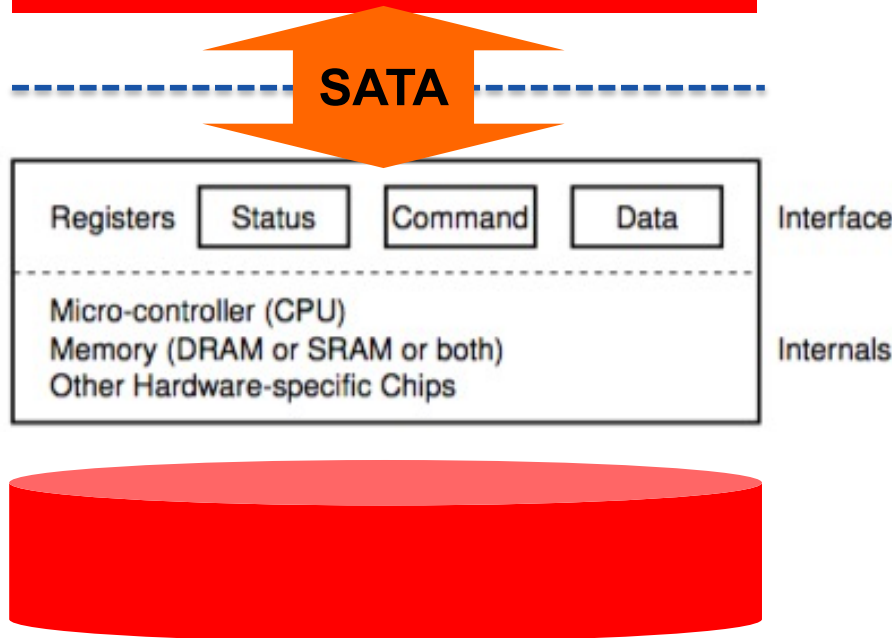
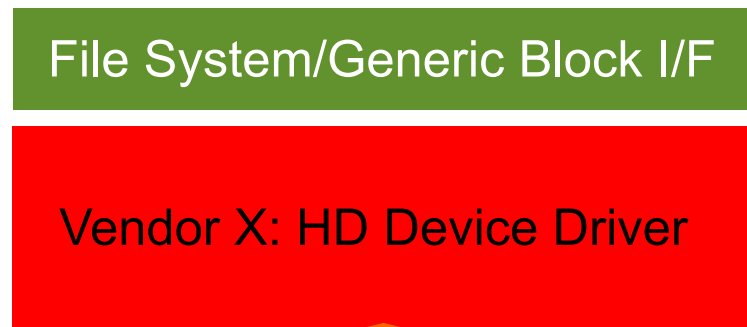
SATA ansluten
HårdDisk i PC





OS

Vi tittar på ett generaliserat HD I/F...



Write block(data, at);

How to access device?
How to wait for ready?
How to transfer data?

Status = { ready, busy }

Command = { read, write }

Data = { (buffer) from, to }

Vänta på ready...
Kopiera data
Skriv kommando
Vänta på klart...



OS

Hur kan programmet utföra I/O operationer...

File System/Generic Block I/F

Vendor X: HD Device Driver

SATA

Registers

Status

Command

Data

Interface

Micro-controller (CPU)

Memory (DRAM or SRAM or both)

Other Hardware-specific Chips

Internals

Write block(data, at);

How to access device?

How to wait for ready?

How to transfer data?

Status = { ready, busy }

Command = { read, write }

Data = { (buffer) from, to }

Speciella instruktioner (x86:in/out), eller minnesmappad I/O (alla andra).



Hur vet OS:et när I/O är klar...

OS

File System/Generic Block I/F

Vendor X: HD Device Driver

SATA

Registers

Status

Command

Data

Interface

Micro-controller (CPU)

Memory (DRAM or SRAM or both)

Other Hardware-specific Chips

Internals

Write block(data, at);

How to access device?

How to wait for ready?

How to transfer data?

Status = { ready, busy }

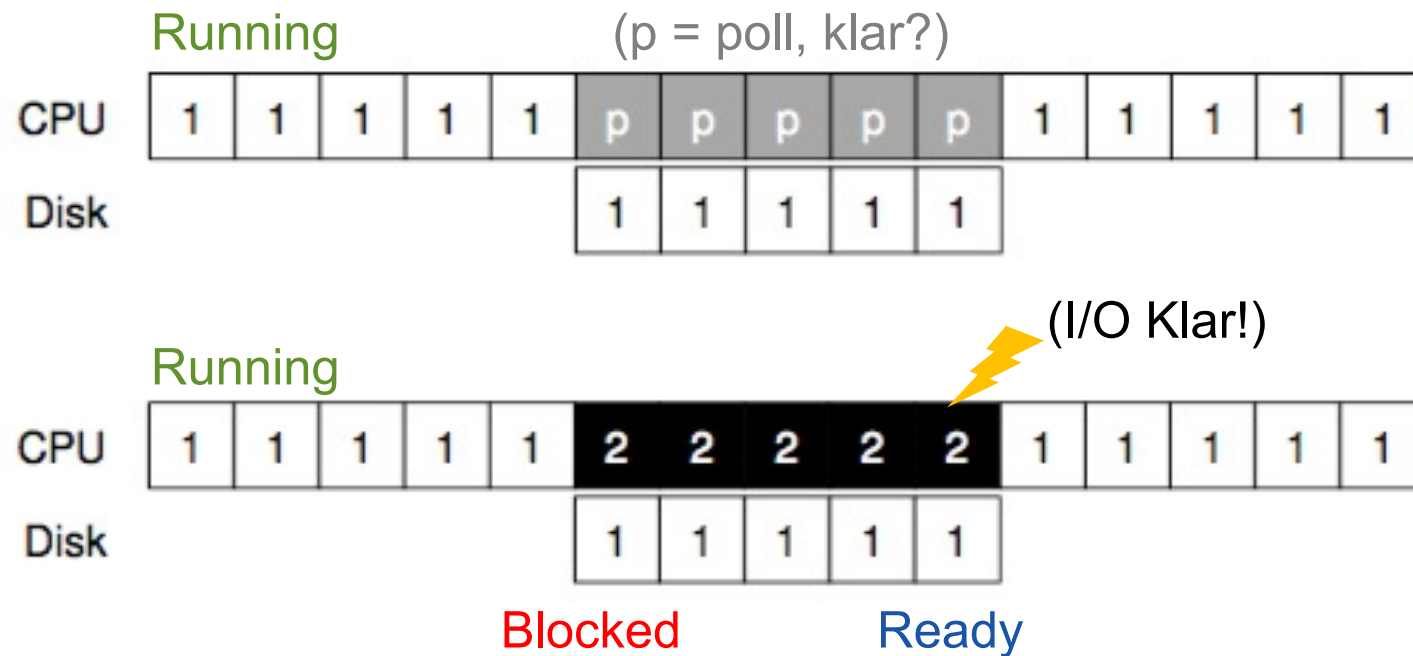
Command = { read, write }

Data = { (buffer) from, to }

Pollad lösning (spinning), alternativt avbrottsstyrd lösning



Hur vet OS:et när I/O är klar...



Märk: Avbrott leder till context switch som tar en hel del tid...
...pollad/hybrid/coalescing kan vara snabbare om kort väntetid!



OS

Hur flyttas data effektivt...

File System/Generic Block I/F

Vendor X: HD Device Driver

SATA

Registers

Status

Command

Data

Interface

Micro-controller (CPU)

Memory (DRAM or SRAM or both)

Other Hardware-specific Chips

Internals

Write block(data, at);

How to access device?
How to wait for ready?
How to transfer data?

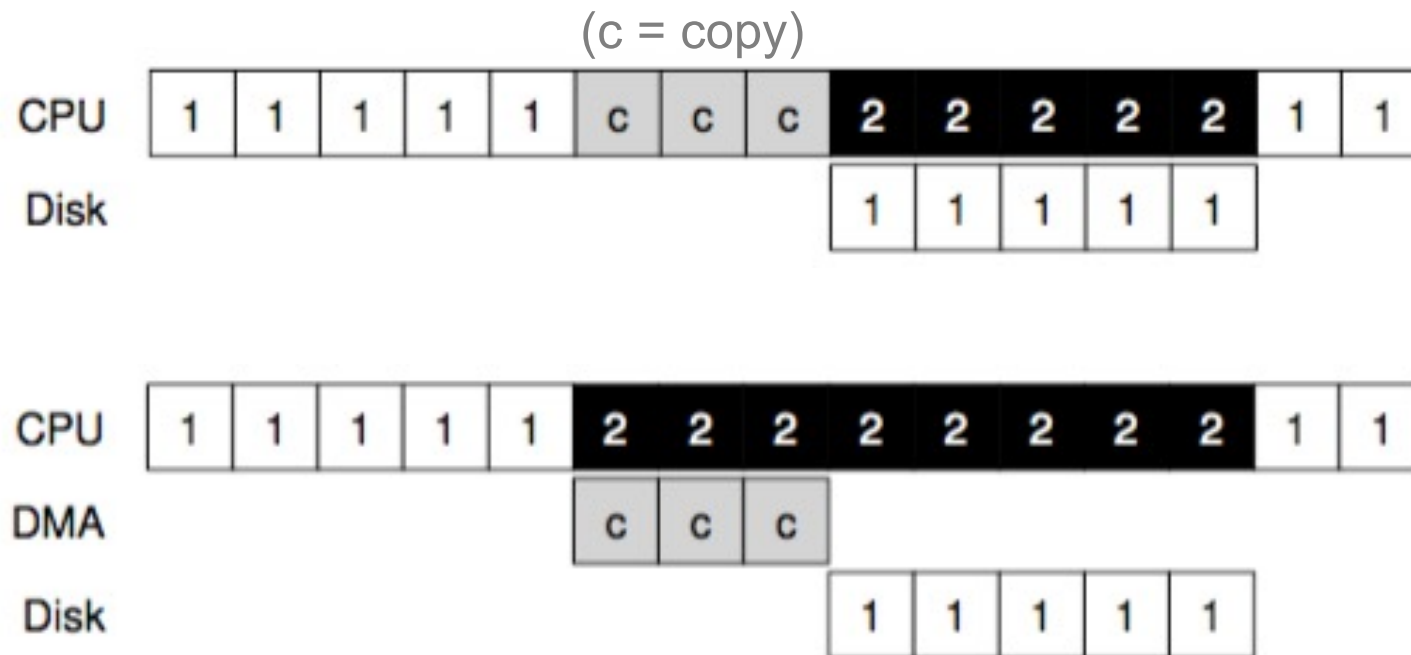
Status = { ready, busy }

Command = { read, write }

Data = { (buffert) from, to }

Det är mycket ineffektivt att processorn flyttar block-data...
...i dag överläts det alltid på Direct Memory Access (DMA) hårdvara!

Hur flyttas data effektivt...



Ska fungera med virtuellt minne och inte fördröja processorn!



Case study: IDE HD

Control Register:

Address 0x3F6 = 0x08 (0000 1RE0): R=reset, E=0 means "enable interrupt"

Command Block Registers:

Address 0x1F0 = Data Port

Address 0x1F1 = Error

Address 0x1F2 = Sector Count

Address 0x1F3 = LBA low byte

Address 0x1F4 = LBA mid byte

Address 0x1F5 = LBA hi byte

Address 0x1F6 = 1B1D TOP4LBA: B=LBA, D=drive

Address 0x1F7 = Command/status

2 Initiera rätt kommando &
3 ge kommandot

Status Register (Address 0x1F7):

7	6	5	4	3	2	1	0
BUSY	READY	FAULT	SEEK	DRQ	CORR	IDDEX	ERROR

4 Hantera avbrott för data

Error Register (Address 0x1F1): (check when Status ERROR==1)

7	6	5	4	3	2	1	0
BBK	UNC	MC	IDNF	MCR	ABRT	TONF	AMNF

BBK = Bad Block

UNC = Uncorrectable data error

MC = Media Changed

IDNF = ID mark Not Found

MCR = Media Change Requested

ABRT = Command aborted

TONF = Track 0 Not Found

AMNF = Address Mark Not Found

5 Hantera/rapportera fel!

1 Wait for
ready!



Case study: IDE HD

```
void ide_rw(struct buf *b) {
    acquire(&ide_lock);
    for (struct buf **pp = &ide_queue; *pp; pp=&(*pp)->qnext)
        ; // walk queue
    *pp = b; // add request to end
    if (ide_queue == b) // if q is empty
        ide_start_request(b); // send req to disk
    while ((b->flags & (B_VALID|B_DIRTY)) != B_VALID)
        sleep(b, &ide_lock); // wait for completion
    release(&ide_lock);
}
```



Case study: IDE HD

```
void ide_rw(struct buf *b) {
    acquire(&ide_lock);
    for (struct buf *pp = ide_queue; pp; pp = pp->next) {
        ;
        *pp = b;
        if (ide_queue == pp)
            ide_start_request(pp);
        while ((b->sector & 0xff) & IDE_BSY) {
            sleep(b);
            release(&ide_lock);
        }
    }
}

static int ide_wait_ready() {
    while (((int r = inb(0x1f7)) & IDE_BSY) || !(r & IDE_DRDY))
        ;
    // loop until drive isn't busy
}

static void ide_start_request(struct buf *b) {
    ide_wait_ready();
    outb(0x3f6, 0); // generate interrupt
    outb(0x1f2, 1); // how many sectors?
    outb(0x1f3, b->sector & 0xff); // LBA goes here ...
    outb(0x1f4, (b->sector >> 8) & 0xff); // ... and here
    outb(0x1f5, (b->sector >> 16) & 0xff); // ... and here!
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));
    if(b->flags & B_DIRTY){
        outb(0x1f7, IDE_CMD_WRITE); // this is a WRITE
        outsl(0x1f0, b->data, 512/4); // transfer data too!
    } else {
        outb(0x1f7, IDE_CMD_READ); // this is a READ (no data)
    }
}
```


Case study: IDE HD

```

void ide_rw(struct buf *b) {
    acquire(&ide_lock);
    for (struct buf *pp = ide_queue; pp; pp = pp->qnext)
        ;
    *pp = b;
    if (ide_queue == pp)
        ide_start_request(b);
    while ((b->flags & B_BUSY) && !ide_wait_ready())
        sleep(b, &ide_lock);
    release(&ide_lock);
}

static int ide_wait_ready() {
    while (((int r = inb(0x1f7)) & IDE_BSY) || !(r & IDE_DRDY))
        ;
    // loop until drive isn't busy
}

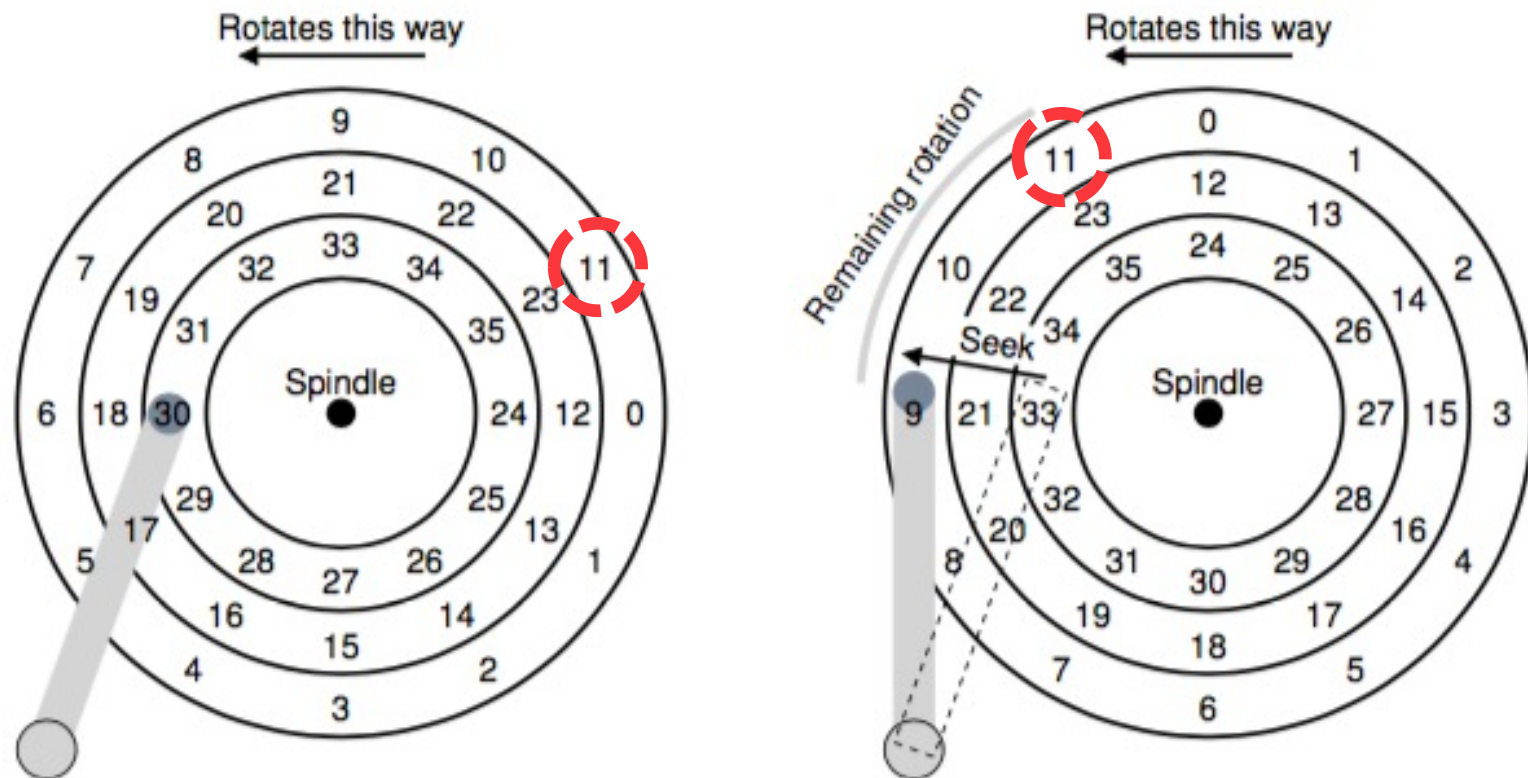
static void ide_start_request(struct buf *b) {
    ide_wait_ready();
    outb(0x3f6, 0); // generate interrupt
    outb(0x1f2, 1); // how many sectors?
    outb(0x1f3, b->sector & 0xff); // LBA goes here ...
    outb(0x1f4, (b->sector >> 8) & 0xff); // ... and here
    outb(0x1f5, 0); // ... and here
}

void ide_intr() {
    struct buf *b;
    acquire(&ide_lock);
    if (!(b->flags & B_DIRTY) && ide_wait_ready() >= 0)
        insl(0x1f0, b->data, 512/4); // if READ: get data
    b->flags |= B_VALID;
    b->flags &= ~B_DIRTY;
    wakeup(b); // wake waiting process
    if ((ide_queue = b->qnext) != 0) // start next request
        ide_start_request(ide_queue); // (if one exists)
    release(&ide_lock);
}

```

Något om hur en Hårddisk fungerar...

Magnetiserbar skiva som roterar med ca 10kRPM, 1 varv 6ms.
Skivan innehåller 100.000-tals tracks, uppdelade i sectors där
varje sektor typisk lagrar 512B som läs/skrivs atomärt.



(Acc, coastin, deacc & settling)

För att läsa/skriva måste läshuvudet flyttas till rätt track (seek-time),
samt att skivan måste rotera till huvudet är i början av rätt sektor.

Något om hur en Hårddisk fungerar...



Överslagsberäkning "accesstid":

Seek-time, typiskt 10ms

wait-track, typ 0.5 varv...

... $0.5 \cdot (60/10000\text{RPM}) = 3\text{ms}$

Läsa 4kByte, typ försumbart

Totalt ca 13ms

En helt otrolig ingenjörsprestation!

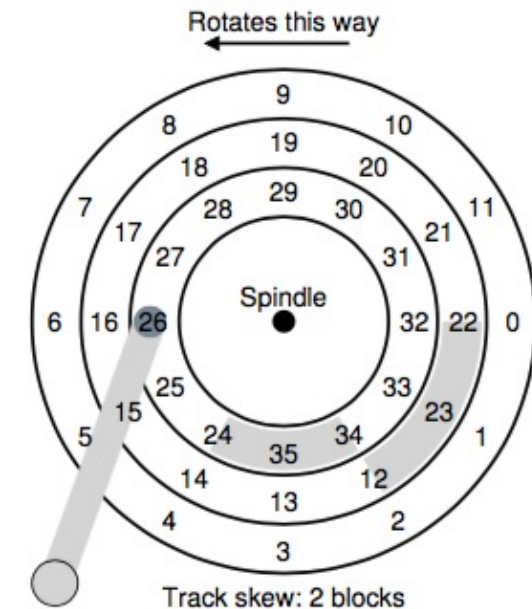


Något om hur en Hårddisk fungerar...

Det går uppenbarar snabbare att läsa en fil som "ryms" i en följd i ett spår...

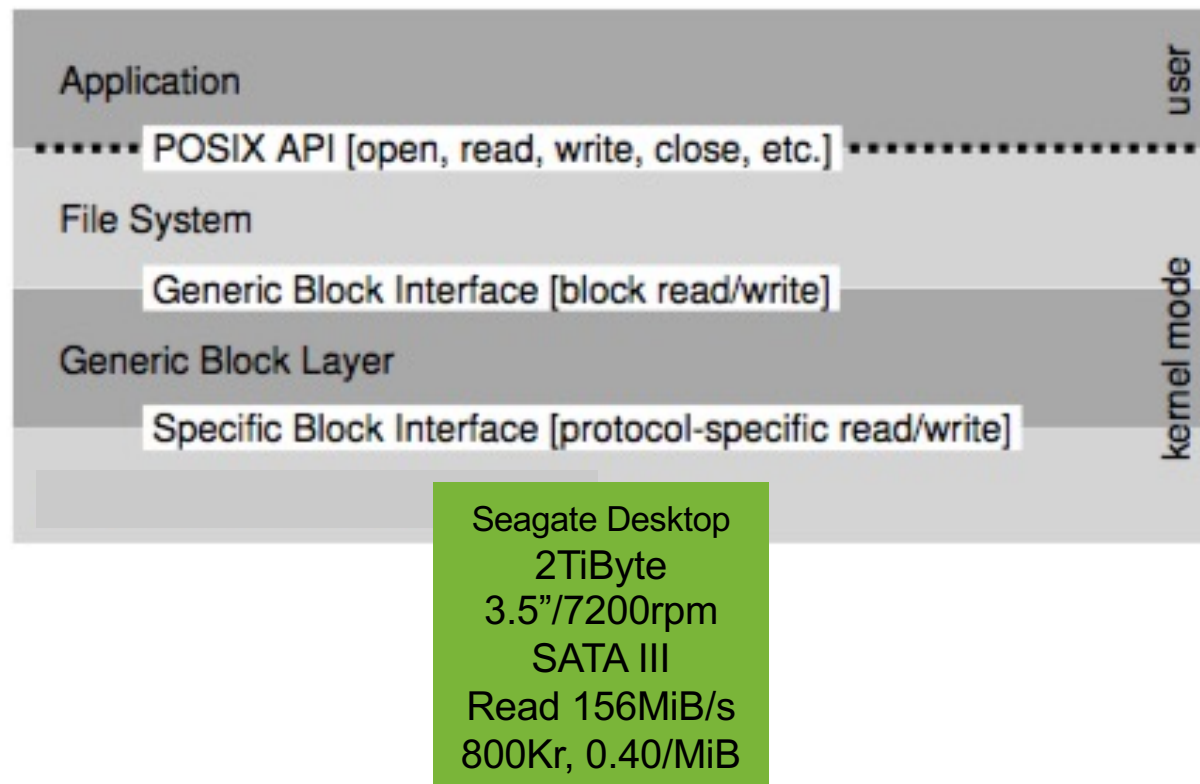
...därför är de yttre spåren "värdefullare" än de inre. Ryms inte en fil i ett spår så är det smart att "fortsätta" en "bit bort" i ett annat spår så huvudet hinner dit under samma rotation (skew)...

...vanligtvis finns också en cache där disken läser in lite mer än vad som efterfrågas, för det blir kanske snart efterfrågat! Cache används också för skrivning men det kräver eftertanke (write back/through).

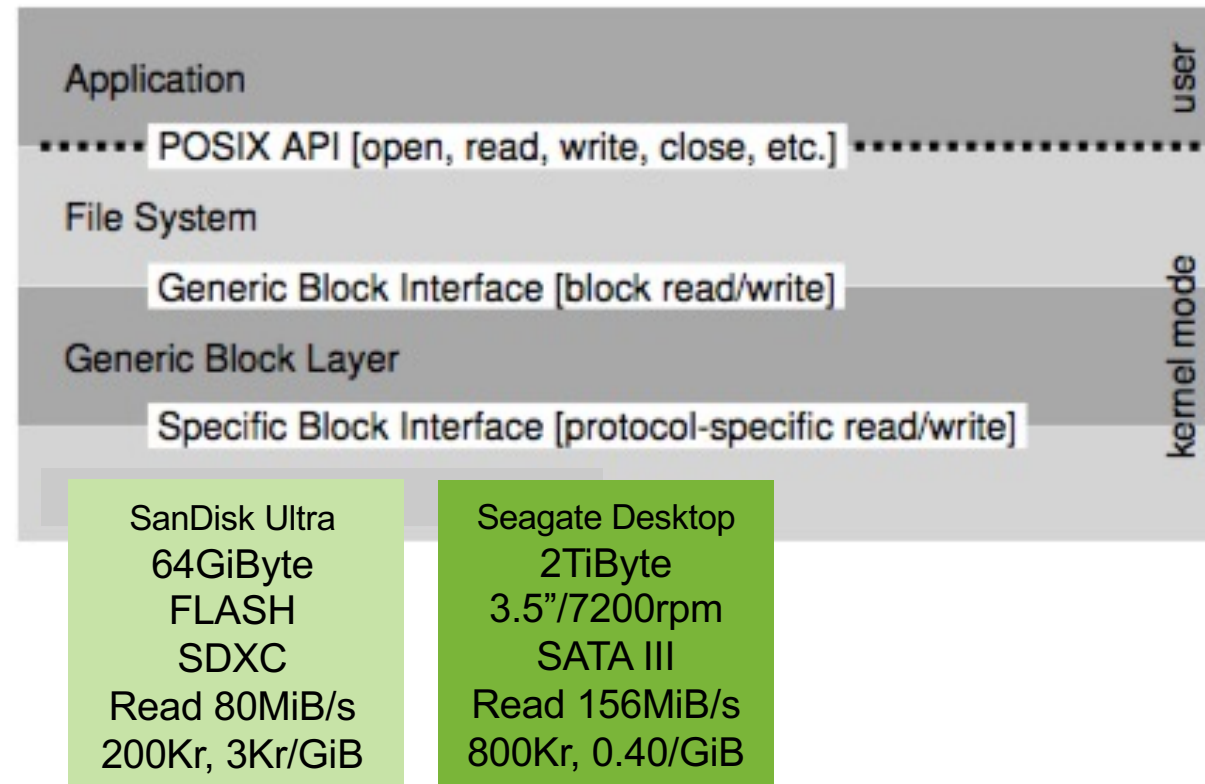


Att skriva nödvändig firmware till en HD är en icke trivial uppgift!

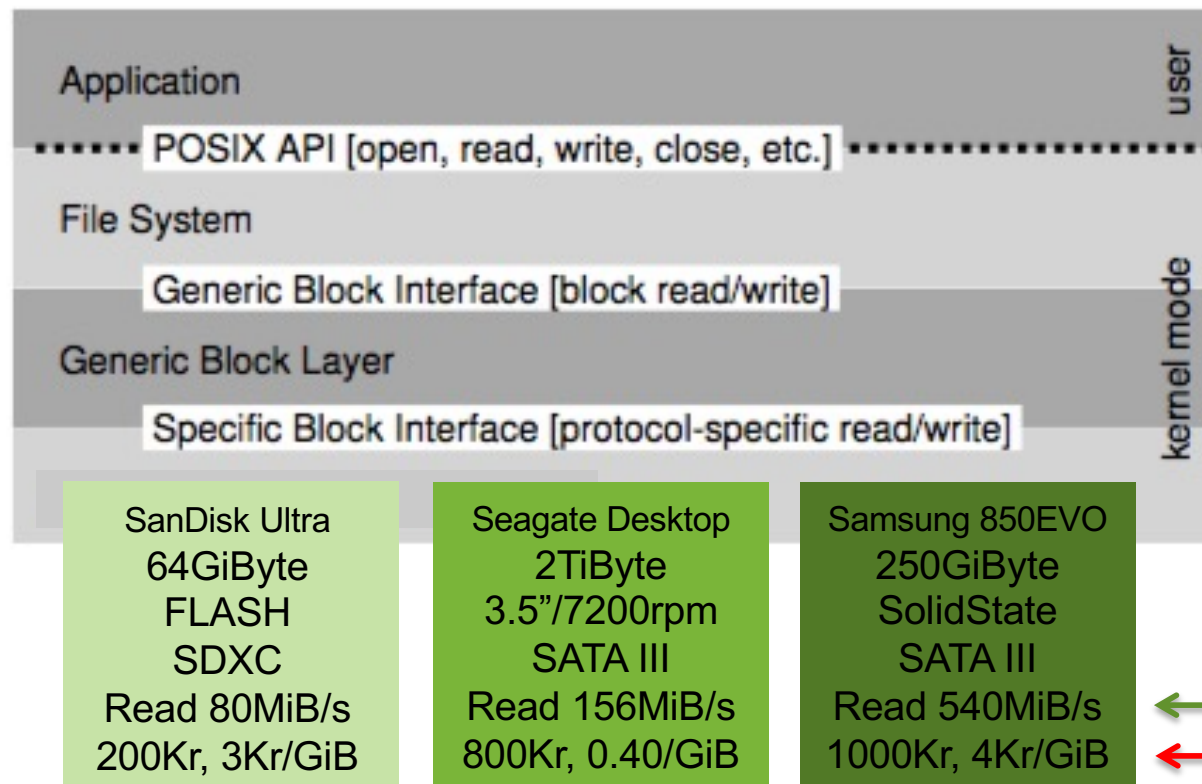
Kan vara väldigt olika...



Kan vara väldigt olika...



Kan vara väldigt olika...





(Inexpensive)

Redundant Array of Independent Disks (RAID)



Använd flera "små" billiga oberoende diskar för att öka prestanda/säkerhet.

6st 2TiB "ser ut som" en 12TiB.

Sprid ut filen på alla 6 diskarna så går den att läsa på 1/6 av tiden.

Sprid ut flera kopior av filen för ökad säkerhet (i bästa fall hot swap).

LaCie 6BIG 12TiB RAID

"Hanteringen" kan ske på applikationsnivån, drivrutinnivån eller den fysiska nivån men syftar alltid till något/flera av målen ovan...



(Inexpensive)

Redundant Array of Independent Disks (RAID)



LaCie 6BIG 12TiB RAID

RAID Level

0: Sprid ut filen på alla diskar

1: Alltid en kopia på en annan disk

2:

3: 2-6 Sprid ut filen på flera diskar,

4: men även paritetsinformation, så

5: att filen kan återskapas!

6:

Det går att kombinera t.ex. 0+1

Blir vanligare och vanligare. Kan finnas i din PC, kan vara ett Storage Area Network (SAN) ansluten till ditt hemnätverk.



Filer...

Ok, spår och sektorer är säkert bra men vi vill att os:et ska erbjuda den abstraktion som vi är vana vid: **filkataloger** och **filer**...

The screenshot shows a file explorer window with a directory tree on the left and a file's properties window on the right. The directory tree shows a hierarchy of folders, including 'Documentation', 'HE1016 Eproj', 'HE1019 EMC', 'HE1028 Mi...atorteknik', 'HE1033 K...ikationsnät', 'HE1034 Te...unikation', 'HE1037 Te...h datakom', 'HE1040 PCB', 'HE1041 uCMedProj', 'HF0000 IntroEngineering', 'HF0010 IntroDatalogi', 'HF1005 Infomet', 'HI110X Exjobb', 'HI1024 C', and 'HI1025 Operativsystem'. The 'HE1037 Te...h datakom' folder is selected. The properties window for 'HE1037F1D' shows details such as 'Typ: Microsoft PowerPoint-presentation', 'Storlek: 2 753 813 byte (2,8 MB på skiva)', 'Plats: Elements • Arbete • Anders • KTH', 'Teching • HE1037 Tele och datakom • Common • Presentations • F1 Fourier', 'Skapad: tisdag 30 oktober 2018 12:48', and 'Ändrad: tisdag 30 oktober 2018 12:55'. The 'Allmänt' tab is active, showing 'Lägg till taggar...' and 'Allmänt:' section. The 'Delning och behörigheter:' section shows a table of permissions:

Namn	Behörighet
OC (jag)	↕ Läs och skriva
staff	↕ Bara läsa
everyone	↕ Bara läsa

En fil erbjuder icke flyktig lagring (persistent storage)

En fil identifieras via sitt namn och en sökväg (path)

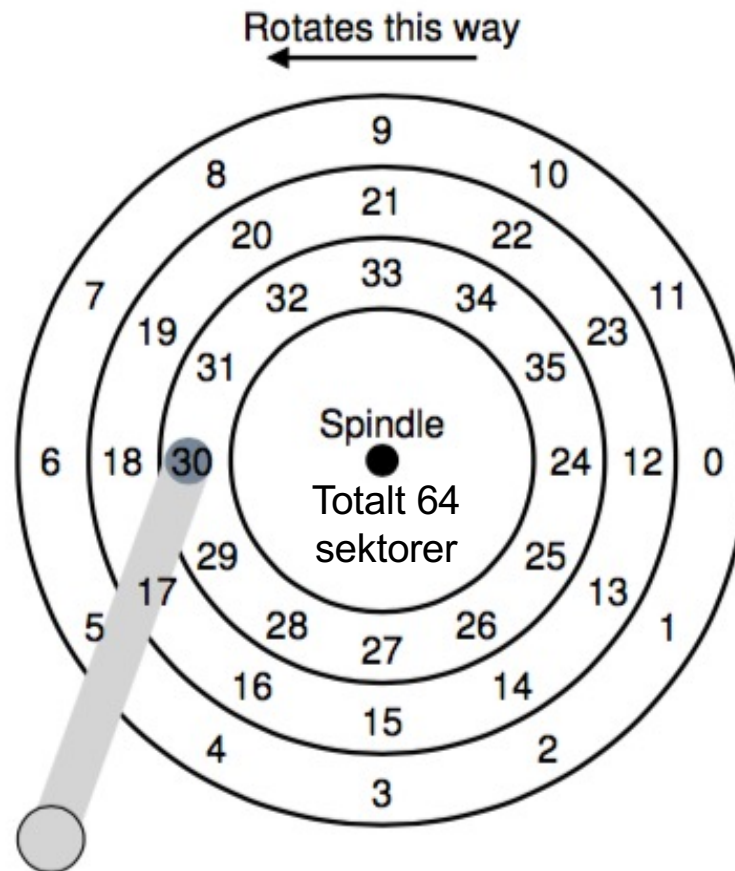
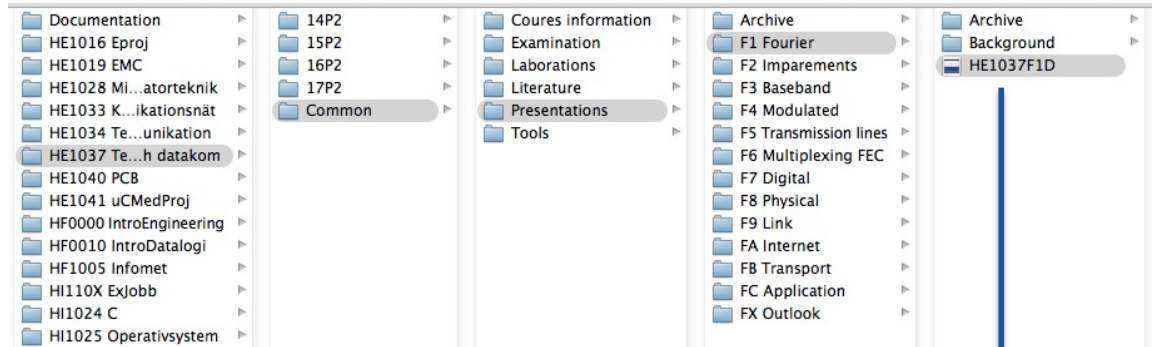
En fil kan vara en delad resurs!

En fil är ur os:ets synvinkel enbart en sekvens av byte

Os:et lagrar dock en mängd meta-data (data om data) om varje fil så som: Storlek, typ, accessrätt., skapad, ägare, senast läst/förändrad, ikon...

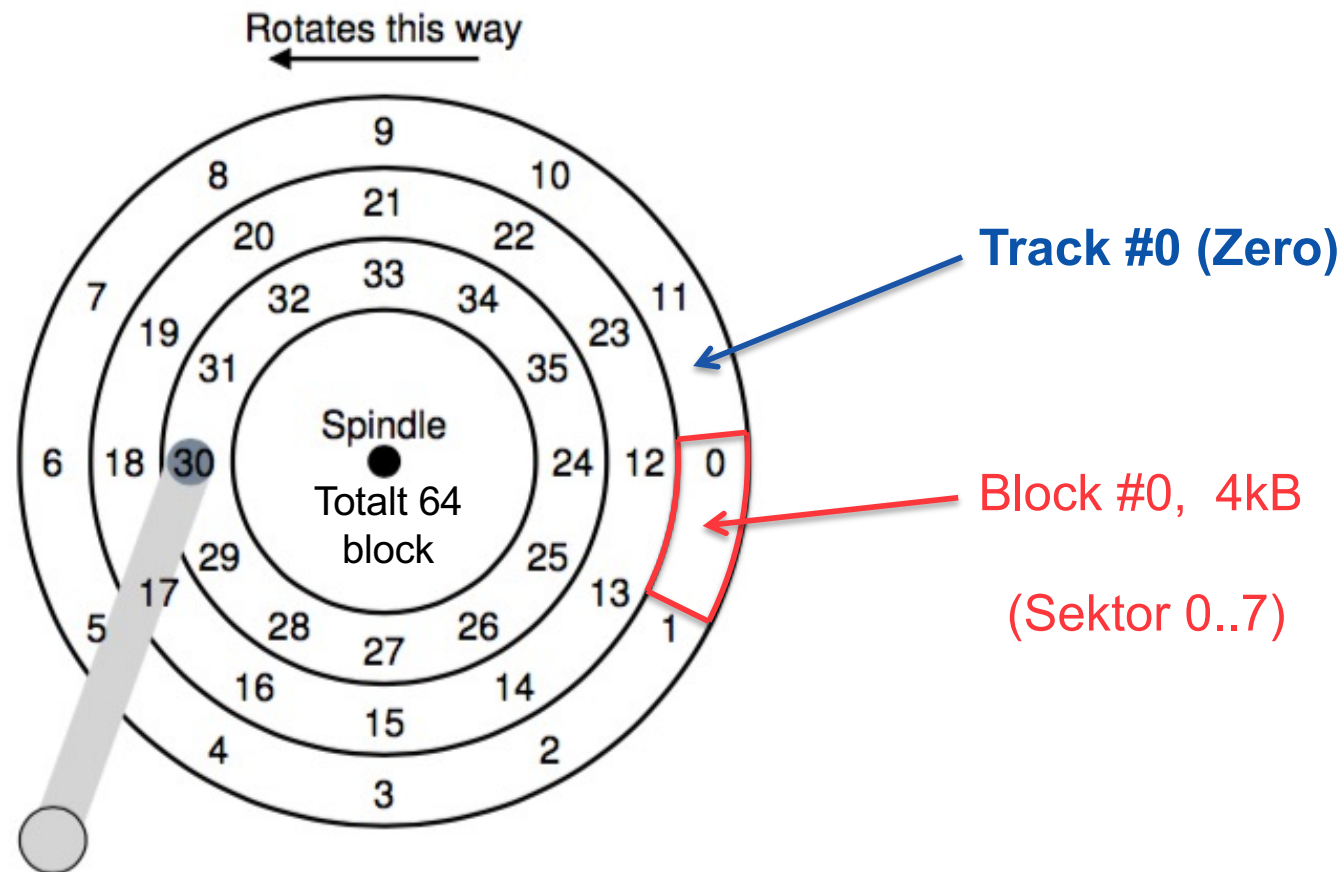


Filer...



Hum, hur går det här till?

Filer...



Hum, vår disk, med spår (tracks) och sektorer (sectors) där varje liten sektor kunde lagra 512Byte, data grupperade i 4kB block...



Filer...

Om vi sen bestämmer:

- 1) En fil tar **alltid** upp **minst ett** block, även om den är mindre.
- 2) All metadata om en fil samlas i en liten datastruktur (256 Byte) som av historiska skäl kallas **inode**.
- 3) Inode posterna måste lagras på disken och det rymmer $4k/256=16$ st/block...
- 4) 5 block kan då hantera $5*16=80$ filer vilket passar hjälpligt "vår" disk.



- A) Det finns alltid en gräns för hur många filer som kan finnas!
- B) Små filer ger ett otroligt svinn (internal fragmentation)!!!



Ok...

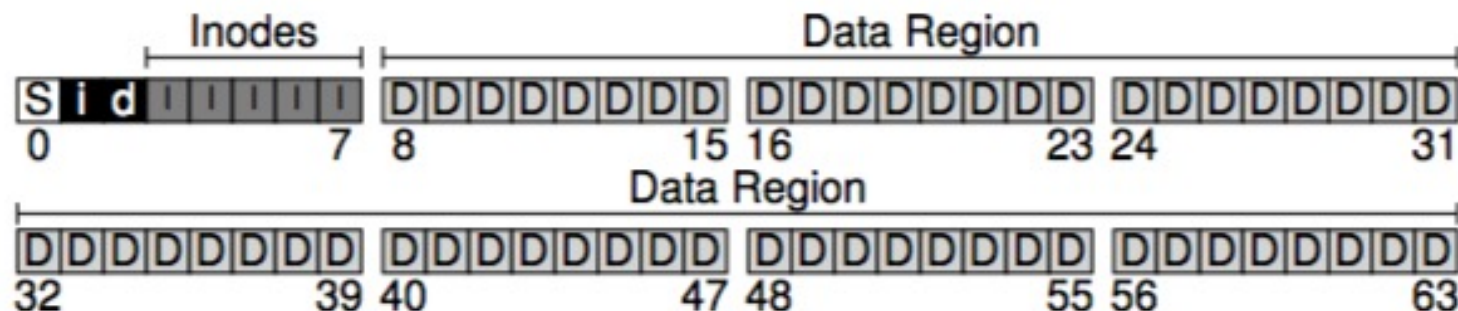
...Vi har en disk med 64 block.

...5 går bort för att lagra inode posterna.

...det behövs också en bitmap som talar om vilka block som är upptagna av filer, och en annan bitmap som talar om vilka inode poster som är använda, de får var sitt block.

...sist, men inte minst, behövs ett block som talar om vilken typ av filsystem skivan har, vart finns inode posterna etc.

...av historiska skäl, lagras informationen på följande sätt:





Observationer...

- ...Att ansluta en hårddisk till ett os kallas att montera den (mount)
- ...OS:et läser då Superblocket för att förstå vilken typ av filsystem som finns på disken, vart inodes finns, etc
- ...Superblocket, bitmappblocken & inode-blocken tillhör alla det yttersta spåret på skivan, track #0, som med andra ord är helt avgörande för diskens funktion (Track Zero error, inte bra!)
- ...Superblocket används också för att indikera om skivan innehåller flera olika filsystem, vilket är möjligt, varje filsystem tillhör då en egen partition (länge begränsat till max 4 partitioner)
- ...Att partitionera en disk är att dela upp den i flera till synes separat filsystem vilket kan vara bra för att skapa testmiljöer, förbättra säkerheten, säkerställa att det finns plats...

Att partitionera en disk är inte trivialt, och raderar vanligtvis disken, då nya bitmapp-/inode-block måste reserveras.

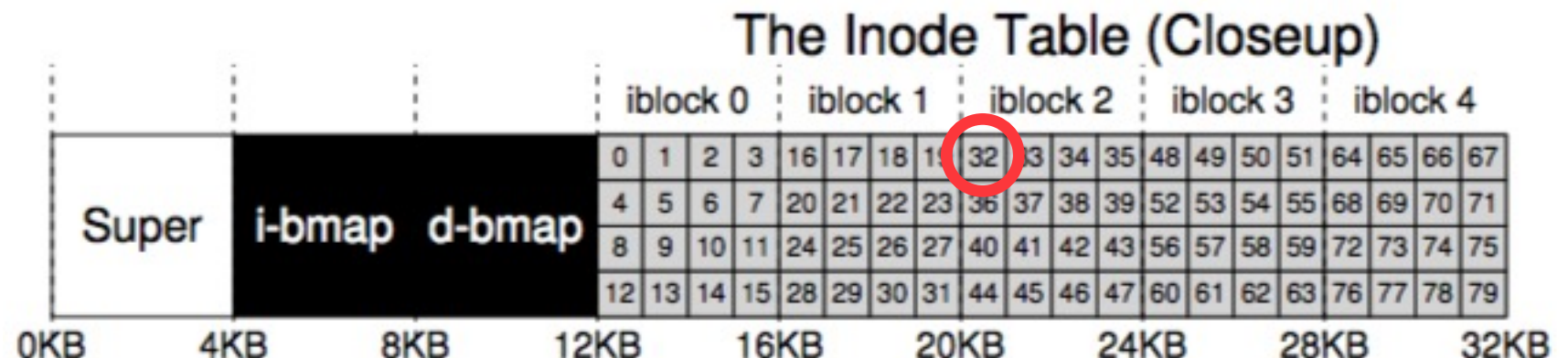


iNode (index (to) Node (descriptor))

Associerat med det läsbara filnamn som en användare anger finns det alltid dolt det index som pekar ut inode-posten i diskens matris av inode-poster. Indexet är filens "verkliga" identifierare!

OS:et kan lätt läsa in rätt inode-post på följande sätt:

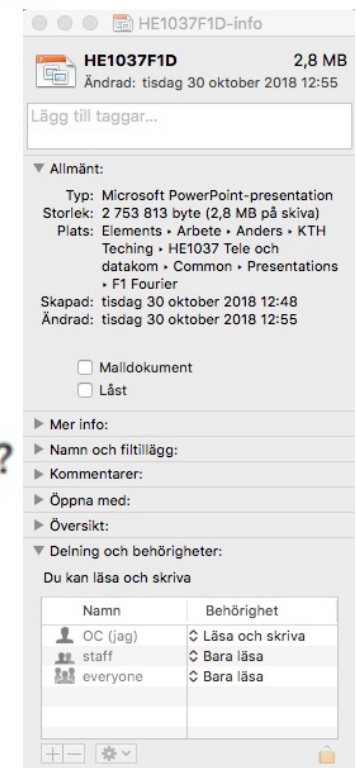
```
blk    = (inumber * sizeof(inode_t)) / blockSize;  
sector = ((blk * blockSize) + inodeStartAddr) / sectorSize;
```



$\text{Blk} = (32 * 256\text{Byte}) / 4\text{kByte} = 2$; $\text{Sektor} = ((2 * 4\text{kB}) + 12\text{kB}) / 512\text{B} = 40$
OS:et ber disken läsa in sektor 40 (och får iNode #32, & #33 på köpet)

iNode (index (to) Node (descriptor))

Size	Name	What is this inode field for?
2	mode	can this file be read/written/executed?
2	uid	who owns this file?
4	size	how many bytes are in this file?
4	time	what time was this file last accessed?
4	ctime	what time was this file created?
4	mtime	what time was this file last modified?
4	dtime	what time was this inode deleted?
2	gid	which group does this file belong to?
2	links_count	how many hard links are there to this file?
4	blocks	how many blocks have been allocated to this file?
4	flags	how should ext2 use this inode?
4	osd1	an OS-dependent field
60	block	a set of disk pointers (15 total)
4	generation	file version (used by NFS)
4	file_acl	a new permissions model beyond mode bits
4	dir_acl	called access control lists



Det som kräver mest eftertanke här är tveklöst hur filens enskilda datablock pekas ut, de behöver inte ligga i någon form av följd!



iNode (index (to) Node (descriptor))

iNode innehåller (kom ihåg att posten max får vara 256Byte):

Array of direct index pointers, i.e. 12, max file size = $12 \cdot 4\text{kB} = 48\text{kB}$

Double-level Index, i.e. 12 direct + 1 indirect = $(12 + 1024) \cdot 4\text{kB} = 4\text{MB}$

(Indirektblocket är allokerat från diskens data-area)

Tripple-level Index, i.e. 12 direct + 1 double indirect = 4GB

Start+Extent, i.e. 6 pairs, 24kB to any size depending on luck!

Linked list, 1 direct (+1 in each data block) = any size (last, rnd acc?)
+ in memory look-up table = maybe ok = File Allocation Table (FAT)

Vanligtvis har en dator otroligt många små filer, men i dag finns ofta också otroligt stora mediefiler, så beslutet är inte enkelt, mer följer...



Filkatalogen...

...den lagras ofta som en fil som helt enkelt innehåller vilka andra filer och kataloger som en mapp innehåller...

...en mapp i filkatalogen är med andra ord en fil som själv har ett inode index som andra mappar kan referera till...

...en mapp har alltid två extra poster, sig själv '.', och sin förälder '..'

inum	reclen	strlen	name
5	12	2	.
2	12	3	..
12	12	4	foo
13	12	4	bar
24	36	28	foobar_is_a_pretty_longname



Detaljer utelämnade i övning #2: LINUX

Hard link (Ln (- in ls))vs Symolic link (Ln -s (l in ls))

Size	Name	What is this inode field for?
2	mode	can this file be read/written/executed?
2	uid	who owns this file?
4	size	how many bytes are in this file?
4	time	what time was this file last accessed?
4	ctime	what time was this file created?
4	mtime	what time was this file last modified?
4	dtime	what time was this inode deleted?
2	gid	which group does this file belong to?
2	links_count	how many hard links are there to this file?
4	blocks	how many blocks have been allocated to this file?
4	flags	how should ext2 use this inode?
4	osd1	an OS-dependent field
60	block	a set of disk pointers (15 total)
4	generation	file version (used by NFS)
4	file_acl	a new permissions model beyond mode bits
4	dir_acl	called access control lists



Free space management (deja vu?!)

Precis som på samma sätt som OS:et managerar tillgängligt heap-minne för en process så kan olika datastrukturer/algoritmer komma på tal för att hantera diskens fria block...

Det är inte ovanligt att en enkel bitmapp används, som i lektionens exempel, där en 1:a betyder att blocket tillhör en fil och en 0:a att blocket inte är allokerat.

Stor vikt läggs på att diskens minne inte ska bli fragmenterat (external fragmentation), så att så stora sekvenser av block som möjligt finns tillgängliga.

Vissa filsystem försöker reservera t.ex. 8 block i följd när en fil skapas så filer upp till 32kB kan manageras så snabbt det går!

Alla OS tillhandahåller en diskfragmenteringsrutin som flyttar runt diskens filers data block för att optimera prestanda – en otroligt riskfylld övning – köp en disk till i stället och kopiera över allt!



Ett konkret exempel!

```
open("/foo/bar", O_RDONLY)
```

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
open(bar)			read ^a		read	read ^b		read		
read()					read			read		
read()					write read				read	
read()					write read					read
					write					

OS:et måste hitta inode för bar, som den bara kan hitta via filkatalogen...
...den läser med andra ord inode för root, som måste ha ett känt id (2)...
...så att den kan ta reda på var root har sitt/sina datablock=katalogen!



Ett konkret exempel!

```
open("/foo/bar", O_RDONLY)
```

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
open(bar)			read a		read c	read b		read d		
					read					
read()					read			read		
					write					
read()					read				read	
					write					
read()					read					read
					write					

I rootkatalogen hittar den inode id för fookatalogen...
...den läser in inode för fookatalogen för att hitta dess datablock...
...så att den kan läsa in datablocken och hitta inode id för bar...



Ett konkret exempel!

```
open("/foo/bar", O_RDONLY)
```

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
open(bar)			read a			read b				
				read c			read d			
					read e					
read()					read			read		
					write					
read()					read				read	
					write					
read()					read					
					write					read

Till sist så kan den i fookatalogen hitta inode id för bar...

...som den läser in, och skapar det som behövs för att OS:et ska kunna returnera en FILE ptr till den process som öppnade filen!



Ett konkret exempel!

`open("/foo/bar", O_RDONLY)` → `read()`

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
open(bar)			read a			read b				
				read c			read d			
				e read	read					
read()				f read	read			read g		
				h write	write					
read()				read	read				read	
				write	write					
read()				read	read					
				write	write					read

När processen sedan vill läsa "nästa" block (vilket det nu är)...

...konsulteras först inode posten att allt fortfarande är ok & data ptr...

...därefter läses blocket in & inode posten uppdateras med t.ex. last rd!



Write/Create

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
create (/foo/bar)		1) read write	read	read		read	read	write 2a)		
			2c) write	2b) read write						
write()	read write				read			write		
write()	read write				write read				write	
write()	read write				write read					write
					write					

För att skapa en ny fil måste en ledig inode allokeras 1) och initieras 2b) samtidigt som dess förälder (parent) uppdateras 2a) & 2c)...



Write/Create

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
create (/foo/bar)		1) read write	read	read		read	read	write 2a)		
			2c) write	2b) read write						
write()	read write 3)				read			write 4)		
write()	read write				write read					write
write()	read write				write read					write

Blir filen "större" pga en write så måste ett nytt datablock allokeras 3)
innan info. skrivs in i det nyallokerade blocket 4) och inode uppdateras.



Det där tar löjligt lång tid -> anv. cache!

Alla moderna OS använder disk cache...

...stora vinster när relaterad data ska läsas...

...när data skrivs, skrivs den också till cachen...

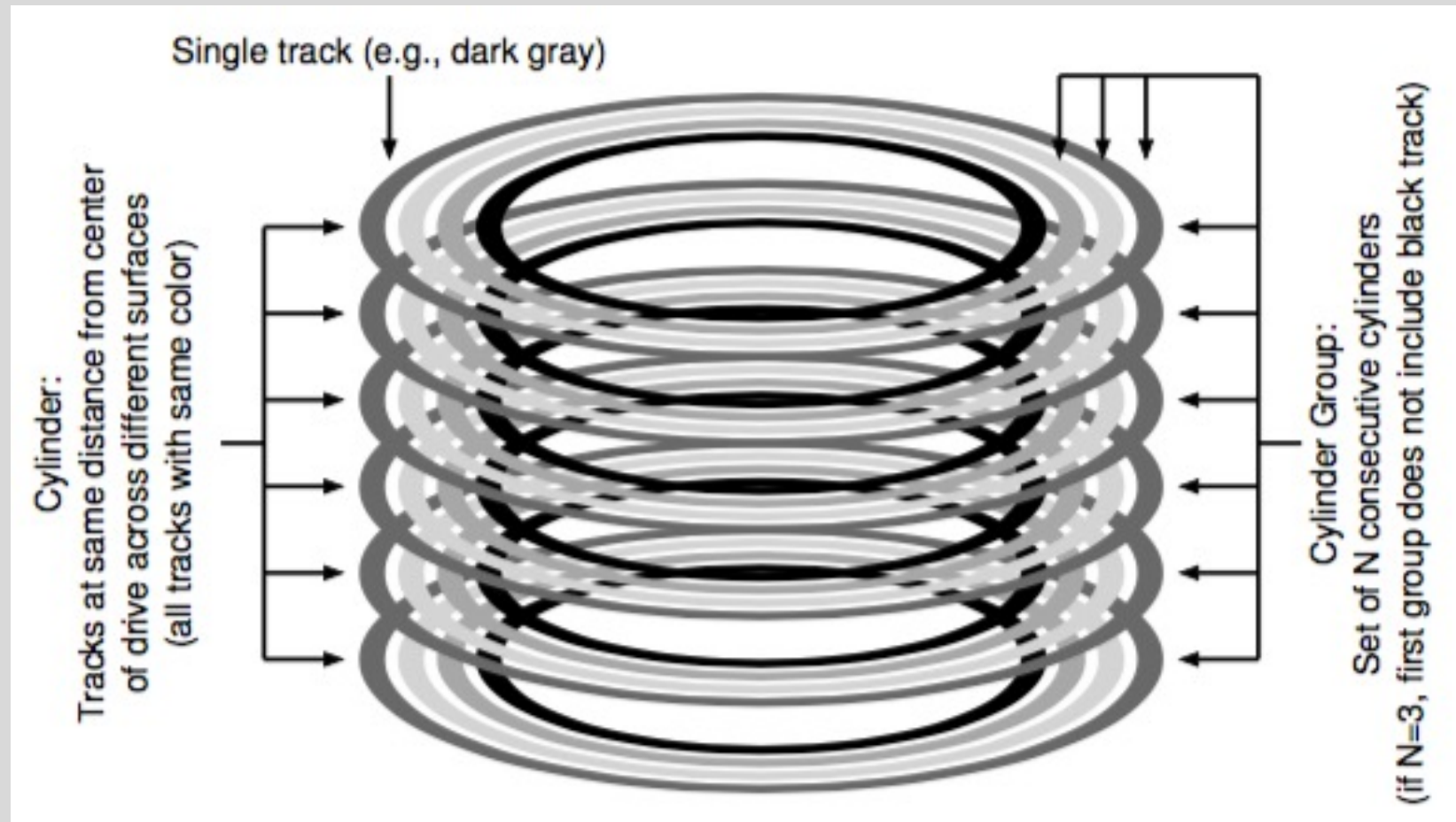
...med lite tur så kan flera "writes" ske samtidigt...

...och vissa undvikas helt, för filen kanske raderas...

...går strömmen kommer data att förloras (5-30 sekunder)!

- 1) DB kräver ofta att en "write" faktiskt utförs (fsync())
- 2) **MATA ALLTID UT EN DISK MED UNMOUNT !!**

Disk aware file systems...



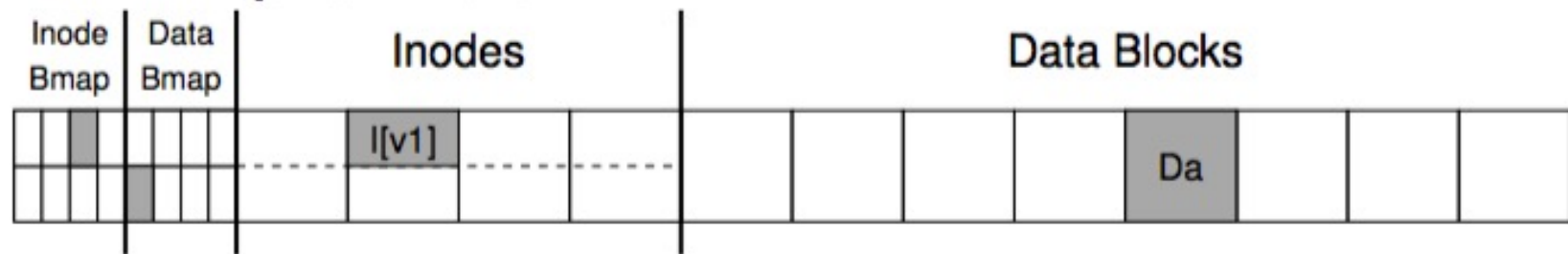
Dagens filsystem försöker inkludera all kunskap den kan få kopplat till den anslutna diskens fysiska konstruktion för att maximera prestanda!



The crash-consistency problem!

```
owner      : remzi
permissions : read-write
size       : 1
pointer    : 4
pointer    : null
pointer    : null
pointer    : null
```

”Append”

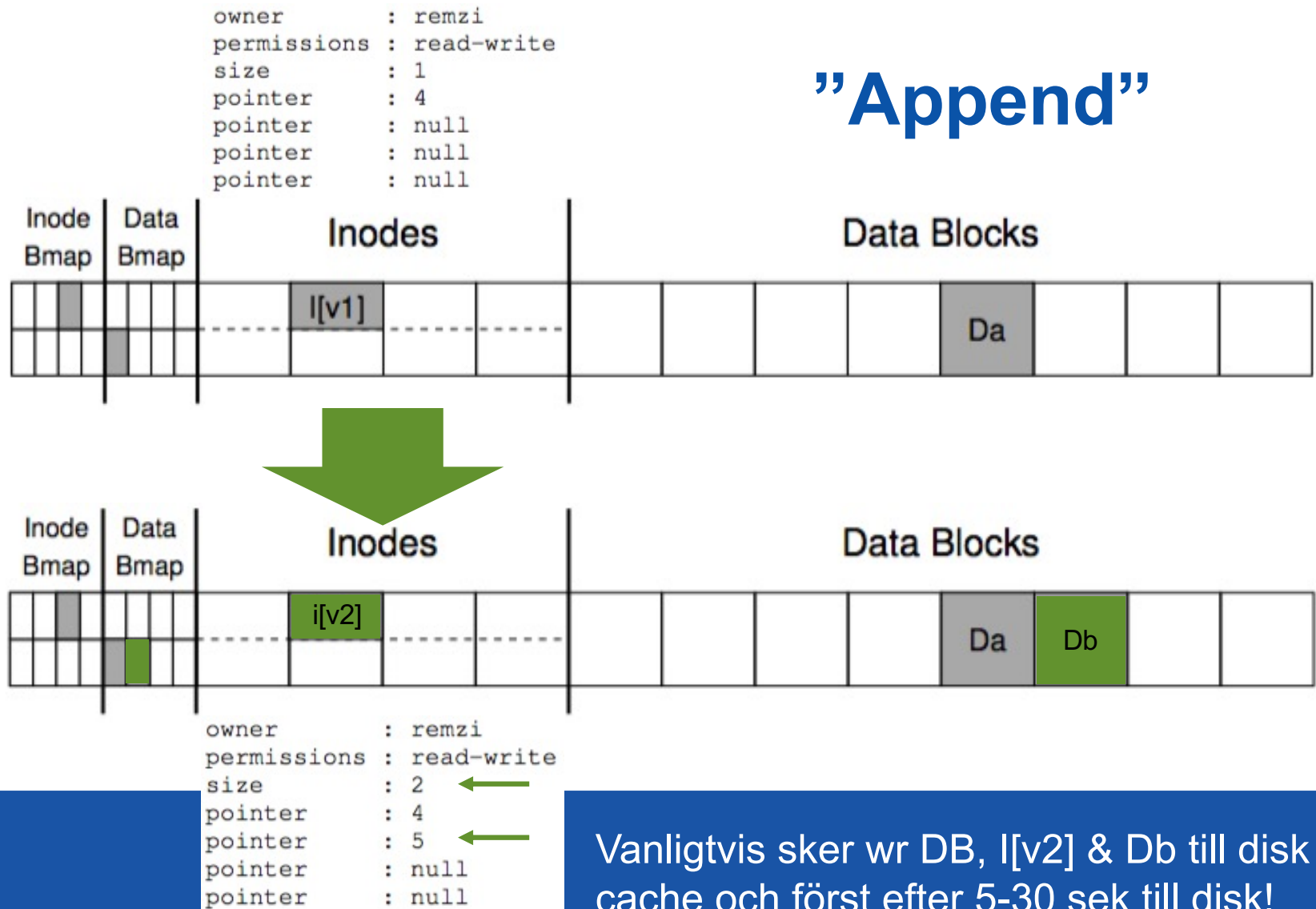


Vårt c-program öppnar filen för "append" och ska lägga till 4k data...



The crash-consistency problem!

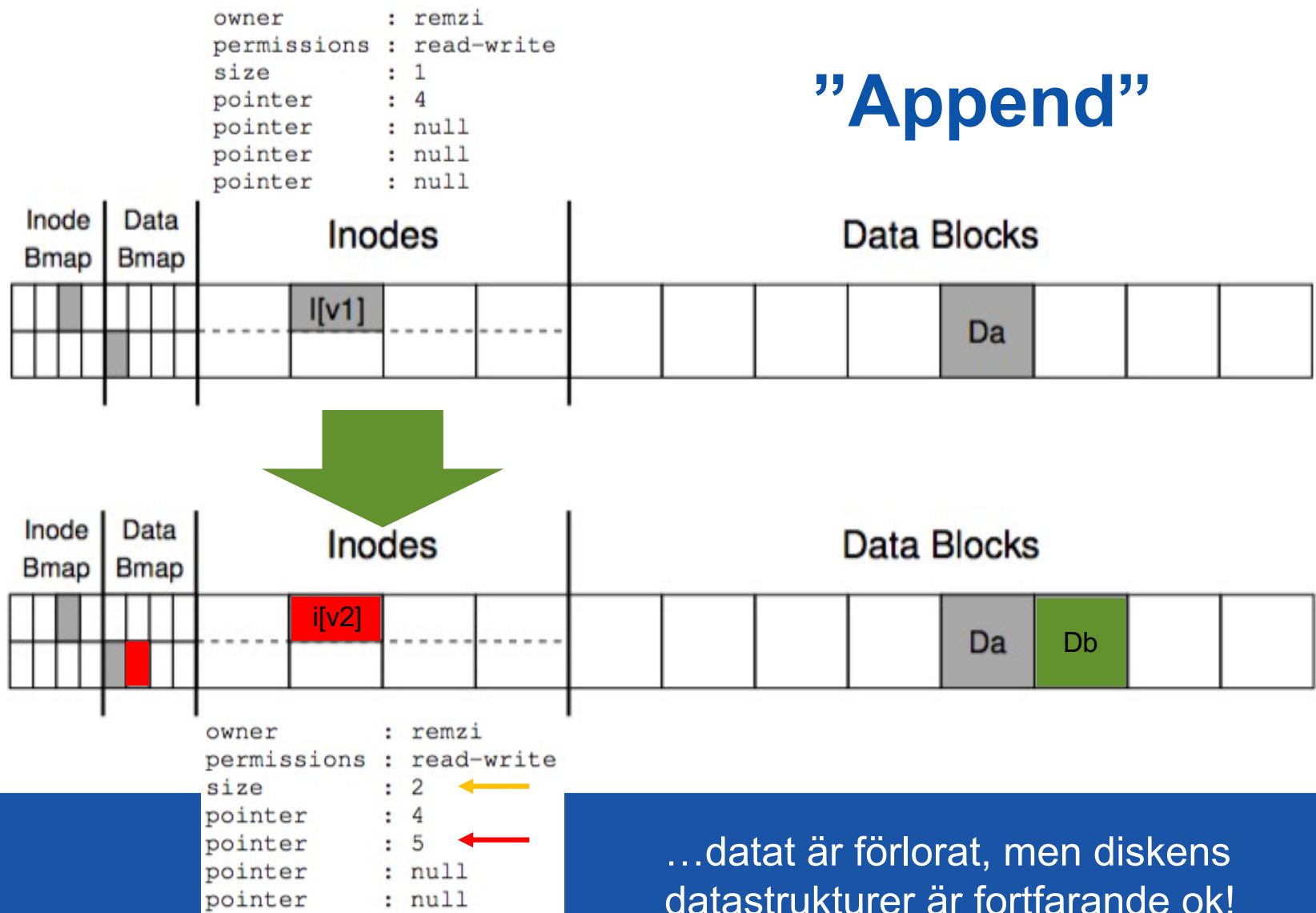
”Append”





Om bara Db-blocket blir skrivet...

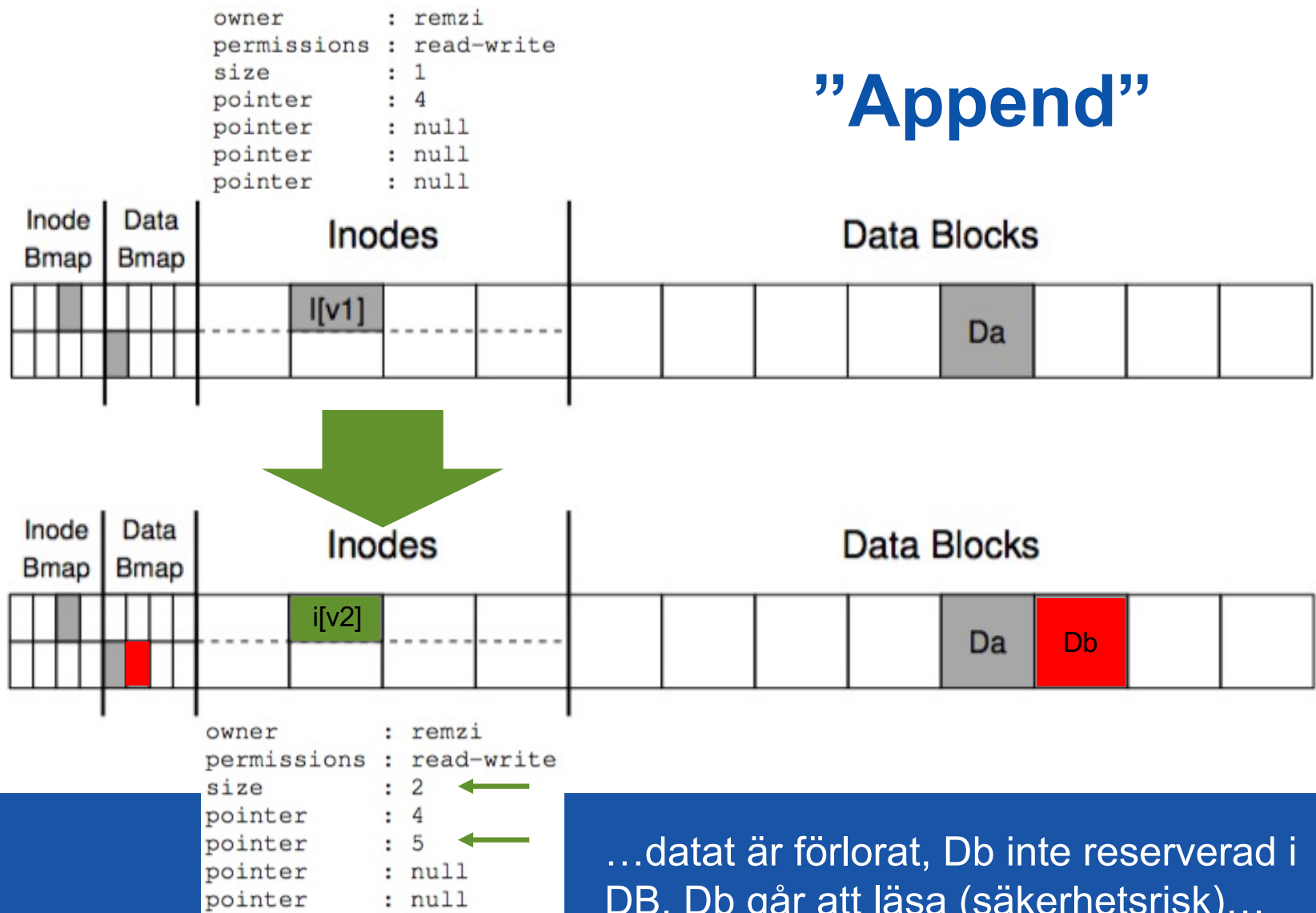
”Append”



...datat är förlorat, men diskens datastrukturer är fortfarande ok!

Om bara iNode-blocket blir skrivet...

"Append"

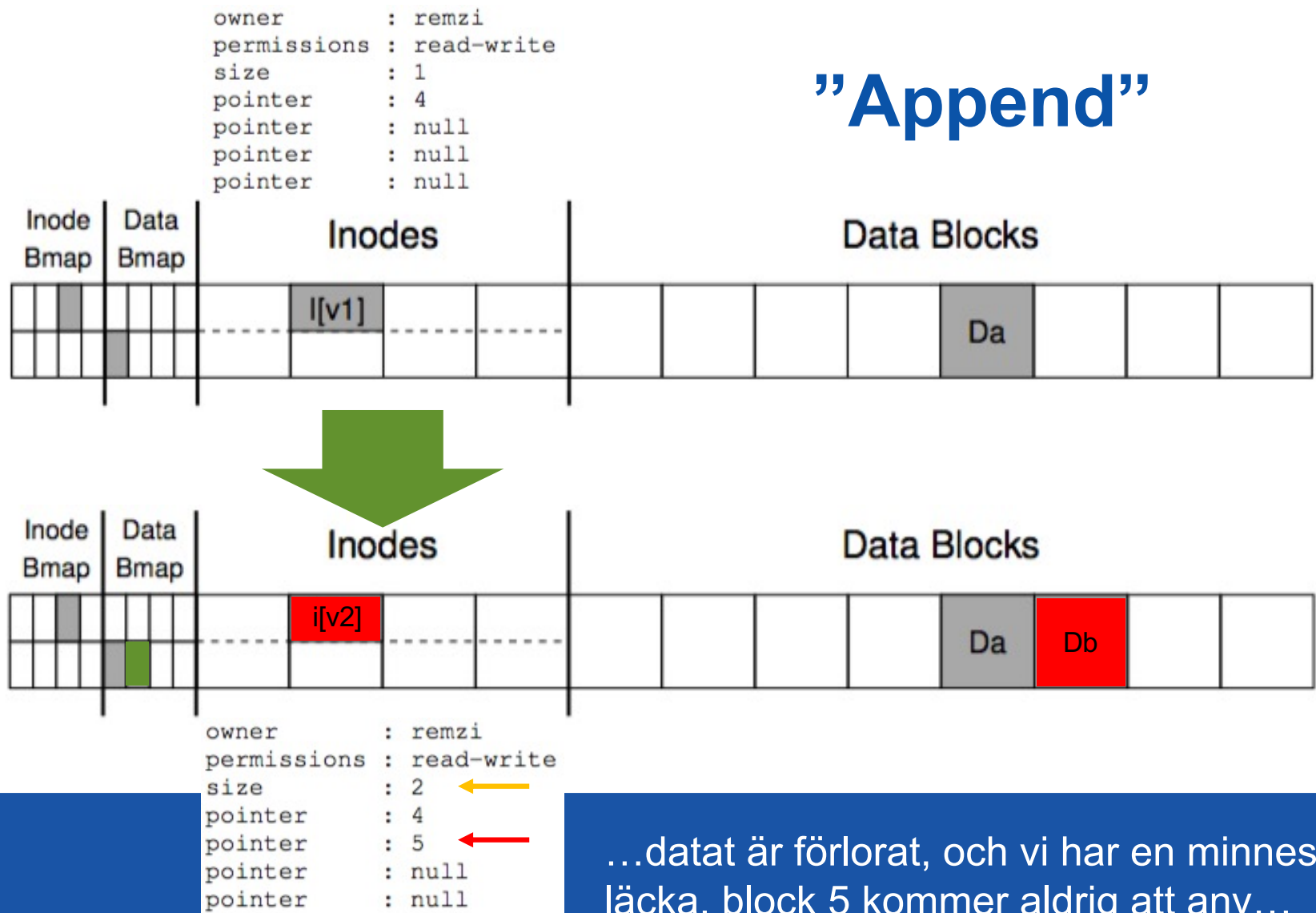


...datat är förlorat, Db inte reserverad i DB, Db går att läsa (säkerhetsrisk)...



Om bara Bmap-blocket blir skrivet...

”Append”



...datat är förlorat, och vi har en minnesläcka, block 5 kommer aldrig att anv...



Om bara XXXXX blir skrivet...

(XXXXX = Godtycklig kombination av allt tänkbart elände)

Problemet är att vi skulle vilja att de 3 skrivoperationerna var atomära men till skillnad från i processfallet så kan vi inte säkerställa det – det går bara att skriva en sektor åt gången – och strömmen kan brytas vid en godtycklig tidpunkt!

Ett filsystem måste helt enkelt kunna hantera en crash!

I praktiken sker det främst på 3 sätt...



A) Städa vid mount...

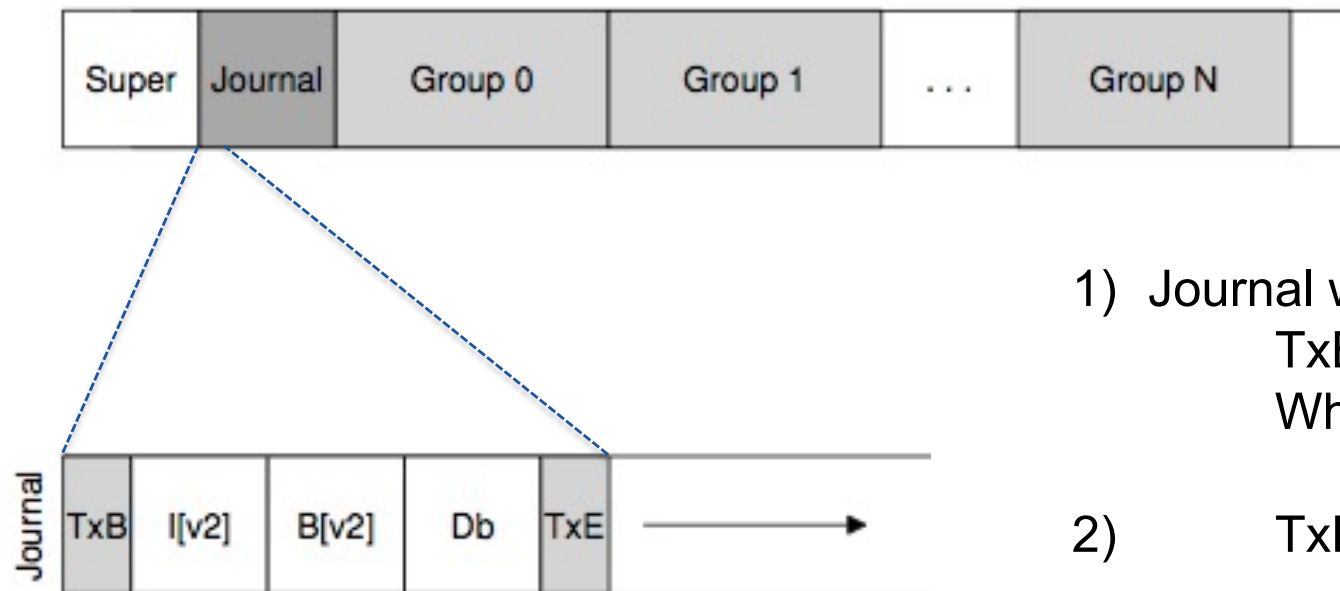
...vid mount, innan någon får använda disken, kan os:et gå igenom en disks datastrukturer och efter förmåga försöka reparera uppenbara fel...

- 1) Undersök superblocket, om det inte ser ok ut, använd en kopia (i dag finns ofta flera kopior utspridda på disken).
- 2) Undersök alla iNode:s, och matcha resultatet mot både iNode- och data bit map. iNode som inte kan räddas, ta bort den och dess allokeringar. Ta bort datablocks allokeringar som inte refereras. Ta bort datablockspekare som inte finns allokerade. Kopior/Dåliga/Katalogfel...

(Linux: **fsck** (File System Checker))

Inget data kan räddas, men diskens datastrukturer återställs!

B) Journal (Write-Ahead Logging)



1) Journal write
TxB with ID
What to do

2) TxE with ID

3) Checkpoint
Exexute...

Eftersom Journal write inte kan ske atomärt delas den i två steg...
...TxB+What to do skrivs i "någon" ordning, säkert sist TxE!



B) Journal (Write-Ahead Logging)

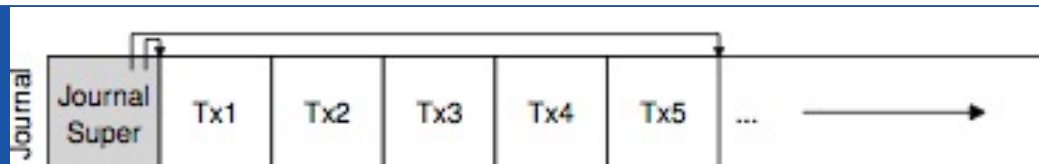
Recovery...

...när filsystemet ska monteras efter en krasch:

- 1) Skippa transaktioner utan korrekt TxE (ofullständiga!)
- 2) Utför alla fullständiga transaktioner (ok med ny krasch!)

(Många filsystem buffrar förändringar i ram för att sedan skapa en komplex transaktion som utför flera uppdateringar.)

Processen måste också ha ett 4:e steg där en korrekt utförd transaktion tas bort från transaktionsloggen när den är utförd. Journal Super block med r/w-pekare till cirkulär lista:



+ snabb recovery
- Tar tid norm. drift



B) Journal (Write-Ahead Logging)

(Only) Metadata Journaling (aka ordered)...

...datablock wr properly!



1. **Data write:** Write data to final location; wait for completion (the wait is optional; see below for details).
2. **Journal metadata write:** Write the begin block and metadata to the log; wait for writes to complete.
3. **Journal commit:** Write the transaction commit block (containing TxE) to the log; wait for the write to complete; the transaction (including data) is now **committed**.
4. **Checkpoint metadata:** Write the contents of the metadata update to their final locations within the file system.
5. **Free:** Later, mark the transaction free in journal superblock.

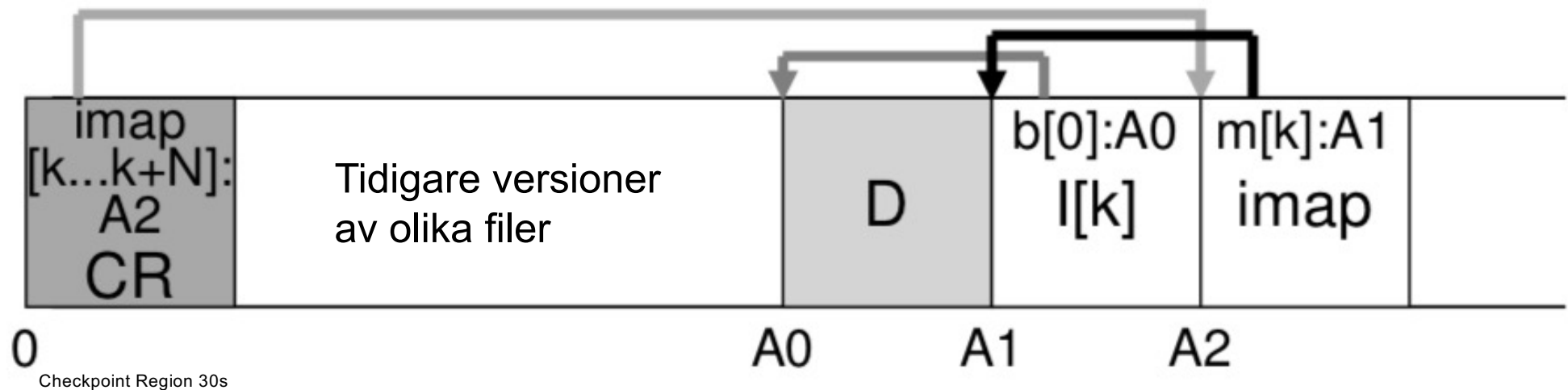
Specialfall...

...delete, som orientering.

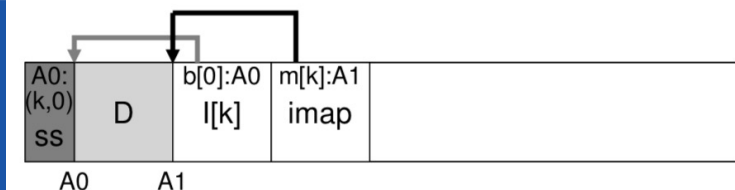
C) Log-structured file systems (43)

Hum...

- Tillgången till (snabbt) DRAM ökar ständigt
 - Stor tidsskillnad mellan sekvensiell och slumpvis access
 - Väldigt många steg för att utföra vanliga uppgifter
 - Ej optimerat för RAID
- ...om vi måste (cache) skriva till HD, uppdatera inte – skriv nytt!



Garbage Collection!





Begrepp

**Persistsens, Storage
Device Drivers**

**Memory mapped I/O vs.
Dedicated Instructions**

**Polled vs. Interrupt driven I/O
Direct Memory Access (DMA)**

Seek vs. Rotation Time.

RAID

File vs. Directory, (Un-)Mount

Track (#0), Block & Sector

iNode, Superblock, Bit Map Block

. vs. ..

Disk Aware File Systems

Crash-consistency problem

fsck vs. Journaling (write-ahead logging)

Log-structured file systems





Historik

- 1810xx AC 0.5 Skapad, övergripande struktur.
- 181105 AC 0.8 Allt utom journaling & logging ok.
- 181112 AC 0.9 Log file system kvar.
- 190120 AC 1.0 Komplett.
- 200211 AC 1.1 Mindre justeringar.